

AlgLinWithSymPy

November 18, 2024

1 Introduction au calcul symbolique avec SymPy

L'objectif de cette feuille de travail est d'introduire aux fonctionnalités de la bibliothèque de calcul symbolique SymPy. Essentiellement celles qui concernent l'algèbre linéaire.

```
[5]: from sympy import *
```

1.1 Symboles et expressions

1.1.1 Declaration de symboles

La bibliothèque `sympy` permet de faire ce qu'on appelle du calcul symbolique, par opposition à `numpy/scipy` qui fait du calcul numérique. Concrètement, elle est capable de manipuler des symboles qui ont un sens mathématique, comme des noms de variables ou de fonctions, comme on le ferait à la main. Avant d'utiliser un symbole, il faut le déclarer en tant que tel sinon Python se plaindra d'une variable inconnue. L'une des fonctions `sympy` pour le faire est `symbols`.

```
[1]: x+y
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[1], line 1  
----> 1 x+y  
  
NameError: name 'x' is not defined
```

Dans le code précédent, Python se plaint de variables non définies. Ce qui est normal puisque les noms 'x' et 'y' n'ont fait l'objet d'aucune affectation. Mais alors comment faire du calcul symbolique dans ces conditions ? Comment définir un polynôme avec une ou plusieurs indéterminé par exemple ?

```
[125]: symbols('x')+symbols('y') # S('x') fait la même chose.
```

```
[125]: x + y
```

```
[126]: symbols('x')**2+2*symbols('x')+1
```

```
[126]: x2 + 2x + 1
```

```
[5]: factor(_) # dans une session interactive, _ fait référence au résultat de ↵
      ↪ l'instruction précédente.
```

```
[5]: (x + 1)2
```

Il serait trop lourd de toujours utiliser des noms de variables de cette manière. En fait il suffit d'affecter à `x` le résultat de `symbols('x')` pour pouvoir utiliser ensuite `x` exactement comme si on fait appel à `symbols('x')`.

```
[7]: x=symbols('x')
     factor(x**3-3*x**2+3*x-1)
```

```
[7]: (x - 1)3
```

Rien n'oblige que le nom de la variable soit le même que le nom du «symbole»

```
[127]: om=symbols('omega') # utiliser le nom de la variable et non celui du symbole ↵
      ↪ dans les expressions qui utilisent le symbole
      expand((om+1)*(om-1))
```

```
[127]: ω2 - 1
```

On peut procéder à une affectation par lot (pour déclarer tous les symboles de la session par exemple)

```
[7]: x,y,z,t=symbols('x y z t') # declare les noms x, y, z et t comme des variables ↵
      ↪ symboliques.
```

Et même utiliser des «indices»

```
[ ]: symbols('m1:5') # crée un tuple de symboles
```

```
[ ]: (m1, m2, m3, m4)
```

```
[9]: m=symbols('m:10') ; m # m sera un tuple, ce qui permet de créer en une seule ↵
      ↪ instruction toute une famille de symboles qui seront par exemple utilisés ↵
      ↪ dans une équation, un système linéaire,...
```

```
[9]: (m0, m1, m2, m3, m4, m5, m6, m7, m8, m9)
```

```
[ ]: m[3] # la variable m3. Utiliser m3 provoquerait une erreur de variable non ↵
      ↪ définie
```

```
[ ]: m3
```

On peut faire des déclarations sur la nature des symboles. Déclarations qui seront ensuite utilisées dans les opérations de transformation d'une éventuelle expression qui utilise les symboles.

```
[11]: sqrt(x**2)
```

```
[11]: √x2
```

```
[12]: x=symbols('x',positive=True) # declare 'x' comme symbolisant un réel positif
```

```
[13]: sqrt(x**2)
```

```
[13]: x
```

Les déclarations possibles : + integer=(True|False) + positive=(True|False) + real=(True|False) + imaginary=(True|False) + complex=(True|False)

1.1.2 Représentation des nombres

Les fractions (nombres rationnels) ont un comportement particulier

```
[14]: sin(1/2*x)
```

```
[14]: sin(0.5x)
```

Ici SymPy a transformé la fraction $1/2$ en un flottant. En fait il n'en est pas responsable. C'est le moteur Python sous-jacent qui a effectué le calcul avant de fournir le résultat aux fonctions SymPy. Si on tient à conserver la fraction $1/2$, il faudra changer sa signification avec la fonction `Rational`

```
[15]: sin(Rational(1/2)*x)
```

```
[15]: sin( $\frac{x}{2}$ )
```

```
[16]: sin(Rational(1)/2*x) # ici Rational(1) empêche Python de «voir» le nombre 1/2.
```

```
[16]: sin( $\frac{x}{2}$ )
```

La fonction `S` peut être aussi utilisée dans de tels situations

```
[108]: sin(S(1)/2*x)
```

```
[108]: sin( $\frac{x}{2}$ )
```

Certains nombres sont au contraire directement représentés par des symboles et il faut à l'inverse forcer leurs évaluation en des nombres flottants avec la méthode `evalf` si on veut les remplacer par des valeurs approchées. Les symboles `pi`, `E` et `I` en font partie.

```
[130]: print(f"pi = {pi}, sin(pi) = {sin(pi)}, cos(pi) = {cos(pi)}, exp(I*pi) =  $\square$   
      ↪ {exp(I*pi)}, exp(I*pi/2) = {exp(I*pi/2)}")
```

```
pi = pi, sin(pi) = 0, cos(pi) = -1, exp(I*pi) = -1, exp(I*pi/2) = I
```

```
[87]: print('pi ~ ',pi.evalf(50), 'E ~ ',E.evalf(50))
```

```
pi ~ 3.1415926535897932384626433832795028841971693993751 E ~  
2.7182818284590452353602874713526624977572470937000
```

```
[90]: acos(0) # arccos(0)
```

```
[90]:  $\frac{\pi}{2}$ 
```

1.1.3 Expressions SymPy

N.B : dans une session interactive le caractère `_` fait référence au résultat de la dernière instruction exécutée.

Presque tout est considéré comme une “expression” dans SymPy. C’est une construction qui peut utiliser des nombres, des opérateurs et des symboles qui peuvent eux même représenter des objets de type différents (des noms de fonctions, de listes, de tableaux, de matrices,...)

```
[114]: x,y,z,t=symbols('x y z t')
      P=x**2+y*z*t ; P # on peut utiliser "l'expression" P sans que cela ne
      ↪déclenche une erreur pour variable inconnue
```

```
[114]: t*y*z + x2
```

```
[115]: P.subs({x:10, y:1}) # pour faire une substitution de variables dans
      ↪l'expression P
```

```
[115]: t*z + 100
```

```
[118]: P.subs({x:t, y:t, z:2*t})
```

```
[118]: 2*t3 + t2
```

Pour développer une expression factorisée

```
[19]: expand((x-y)*(sin(x)-sin(y))**2)
```

```
[19]: x sin2(x) - 2x sin(x) sin(y) + x sin2(y) - y sin2(x) + 2y sin(x) sin(y) - y sin2(y)
```

et pour au contraire factoriser

```
[20]: factor(_)
```

```
[20]: (x - y) (sin(x) - sin(y))2
```

Pour simplifier une expression

```
[21]: (x**2-y**2)/((x-y)**2*(x+y))
```

```
[21]: 
$$\frac{x^2 - y^2}{(x - y)^2 (x + y)}$$

```

```
[22]: simplify(_)
```

```
[22]: 
$$\frac{1}{x - y}$$

```

Les noms `factor`, `expand` et `simplify` peuvent aussi être utilisés en tant que méthodes Python

```
[109]: ((x**2-y**2)/((x-y)**2*(x+y))).simplify()
```

```
[109]: 
$$\frac{1}{x - y}$$

```

1.1.4 Nom symbolique de fonctions

On peut aussi déclarer un symbole comme étant le nom d'une fonction avec l'option `cls=Function`

```
[119]: f=symbols('f', cls=Function)
```

```
[24]: f(0) # f agit comme une fonction Python
```

```
[24]: f(0)
```

```
[25]: f(x).diff(x) # dérivation symbolique
```

```
[25]:  $\frac{d}{dx}f(x)$ 
```

```
[26]: (x*f(x)**2).diff(x) # sympy connaît toutes les règles de la dérivation
```

```
[26]:  $2xf(x)\frac{d}{dx}f(x) + f^2(x)$ 
```

1.2 Polynômes avec SymPy

Un polynôme peut être créé comme une simple expression utilisant des «symboles» déclarés auparavant. Une expression peut avoir plusieurs indéterminées.

```
[120]: P=x**3-2*x+1 ; Q= x**5-2*x**4+2*x**2-1 ; P1=x**6*y**3-2*x**2*y+1 ;  
      ↪ Q1=x**2*y**2-1  
      display(P,Q,P1,Q1)
```

$$x^3 - 2x + 1$$

$$x^5 - 2x^4 + 2x^2 - 1$$

$$x^6y^3 - 2x^2y + 1$$

$$x^2y^2 - 1$$

Le PGCD de deux polynômes se calcule avec la fonction `gcd` et le PPCM avec la fonction `lcm`. Les deux fonctions ne sont pas réservées uniquement aux polynômes à une seule indéterminée. Elles gèrent aussi les polynômes à plusieurs indéterminées.

```
[121]: D=gcd(P,Q) ; M=lcm(P,Q) ; display(D,M)
```

$$x - 1$$

$$x^7 - x^6 - 3x^5 + 4x^4 + 2x^3 - 3x^2 - x + 1$$

```
[31]: gcdex(P,Q) # PGCD et coefficients de Bezout : retourne un tuple (U,V,D) avec  
      ↪ D=PGCD(P,Q) et UP+VQ=D
```

```
[31]: (-x**3/3 + 2*x/3 - 1/3, x/3 + 2/3, x - 1)
```

```
[32]: D1=gcd(P1,Q1) ; M1=lcm(P1,Q1) ; display(D1,M1)
```

$$x^8y^5 - x^6y^3 - 2x^4y^3 + x^2y^2 + 2x^2y - 1$$

```
[33]: gcdex(P1,Q1) # gcdex n'a pas de sens pour des polynômes à plusieurs
      ↪ indéterminées car K[X,Y] n'est pas un anneau principal
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In [33], line 1
----> 1 gcdex(P1,Q1) # gcdex n'a pas de sens pour des polynômes à plusieurs
      ↪ indéterminées car K[X,Y] n'est pas un anneau principal

File /usr/lib/python3/dist-packages/sympy/polys/polytools.py:5065, in gcdex(f,
      ↪ g, *gens, **args)
    5062     else:
    5063         return domain.to_sympy(s), domain.to_sympy(t), domain.to_sympy(h)
-> 5065 s, t, h = F.gcdex(G, auto=opt.auto)
    5067 if not opt.polys:
    5068     return s.as_expr(), t.as_expr(), h.as_expr()

File /usr/lib/python3/dist-packages/sympy/polys/polytools.py:2566, in Poly.
      ↪ gcdex(f, g, auto)
    2563     F, G = F.to_field(), G.to_field()
    2565 if hasattr(f.rep, 'gcdex'):
-> 2566     s, t, h = F.gcdex(G)
    2567 else: # pragma: no cover
    2568     raise OperationNotSupported(f, 'gcdex')

File /usr/lib/python3/dist-packages/sympy/polys/polymorphisms.py:644, in DMP.
      ↪ gcdex(f, g)
    642     return per(s), per(t), per(h)
    643 else:
--> 644     raise ValueError('univariate polynomial expected')

ValueError: univariate polynomial expected
```

1.2.1 Construction d'un polynôme

Pour profiter des fonctionnalités propres aux polynômes (degré, coefficients,...), il faut déclarer les polynômes comme des objets de type `Poly` et non comme de simples expressions. On définit une instance de l'objet `Poly` avec la fonction `poly` mais on peut aussi convertir une expression prédéfinie en un objet `Poly` grâce à la méthode `as_poly`. À l'inverse la méthode `as_expr` permet de convertir un objet `Poly` en une expression sympy.

```
[ ]: poly(x**2+sqrt(2)*x+1)
```

```
[ ]: Poly(x^2 + x*sqrt(2) + 1, x, sqrt(2), domain = ZZ)
```

Sans précision, tout «symbole» qui n'est pas explicitement dans l'ensemble $\mathbb{Z}$ est considéré

comme une indéterminée ($\sqrt{2}$ en est une dans l'exemple précédent). On peut toutefois préciser l'indéterminée désirée, les autres «symboles» jouant alors le rôle de paramètres.

```
[ ]: poly(x**2+sqrt(2)*x+1,x) # 'x' est la seule indéterminée
```

```
[ ]: Poly(x2 + √2x + 1, x, domain = EX)
```

Là on constate que SymPy a changé de domaine. C'est le corps de base dans lequel sont pris les coefficients. Le mot clé **EX** est utilisé quand sympy n'arrive pas à décider dans quel domaine il doit se placer. On peut en outre préciser un domaine particulier en utilisant l'option **domaine**. Dans cet exemple le domaine qu'on veut est le corps $\mathbb{Q}[\sqrt{2}]$. Il suffit de passer la valeur `QQ(sqrt(2))` à l'option **domain**

```
[86]: poly(x**2+sqrt(2)*x+1,domain='QQ(sqrt(2))')
```

```
[86]: Poly(x2 + √2x + 1, x, domain = Q(√2))
```

Une expression qui contient un Poly est coéorisée en un objet de type Poly

```
[ ]: (x-1)*poly(x**2+sqrt(2)*x+1,domain='QQ(sqrt(2))') #
```

```
[ ]: Poly(x3 + (-1 + √2)x2 + (1 - √2)x - 1, x, domain = Q(√2))
```

La fonction **poly** retourne toujours une écriture développée sous forme ordonnée par degrés décroissants d'un polynôme

```
[ ]: poly((x-1)**4)
```

```
[ ]: Poly(x4 - 4x3 + 6x2 - 4x + 1, x, domain = Z)
```

Il peut être important de récupérer le nom de l'indéterminée utilisée dans un Poly (pour des besoins de programmation par exemple). On y accède à l'aide de la propriété **gen** de l'objet créé (**gen** pour generator).

```
[37]: R=poly(x**2+sqrt(2)*x+1,x) ; R.gen
```

```
[37]: x
```

On peut convertir un objet de type Poly en une expression sympy avec la méthode **as.expr**

```
[38]: R.as_expr() # convertir un 'Poly' en une 'expression'
```

```
[38]: x2 + √2x + 1
```

On peut changer le nom de l'indéterminée dans ce processus de conversion

```
[39]: R.as_expr(z)
```

```
[39]: z2 + √2z + 1
```

1.2.2 Degré, coefficients,...

L'extraction de propriétés d'un «polynôme» ne fonctionne pas avec une simple expression sympy mais seulement avec les objets de type Poly.

```
[ ]: P.degree() #
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In [41], line 1  
----> 1 P.degree() # ne fonction pas avec une simple expression sympy, il faudr  
      ↪ convertir l'expression en un objet `Poly`  
  
AttributeError: 'Add' object has no attribute 'degree'
```

On convertit d'abord P en un objet de type Poly

```
[122]: P=P.as_poly() ; P
```

```
[122]: Poly( $x^3 - 2x + 1, x, domain = \mathbb{Z}$ )
```

```
[123]: P.degree() # le degré
```

```
[123]: 3
```

```
[ ]: P.coefs() # fournit la liste des coefficients mais zappe ceux qui sont nuls
```

```
[ ]: [1, -2, 1]
```

```
[ ]: P.all_coefs() # pour avoir la liste de tous les coefficients, y compris ceux  
      ↪ qui sont nuls
```

```
[ ]: [1, 0, -2, 1]
```

```
[ ]: P.coeff_monomial(x**3) # on peut récupérer un coefficient en particulier avec  
      ↪ la méthode coeff_monomial
```

```
[ ]: 1
```

```
[ ]: P.nth(3) # equivalent de P.coeff_monomial(x**5)
```

```
[ ]: 1
```

La méthode `coeff` est disponible pour les expressions. Elle a le même rôle que `coeff_monomial` pour les polynômes mais ne fonctionne pas avec un Poly.

```
[162]: (x**3+3*x**2-2*x+sqrt(2)*x**2+1).coeff(x**2)
```

```
[162]:  $\sqrt{2} + 3$ 
```

Il faut toutefois penser à développer l'expression (inutile avec un Poly)

```
[163]: ((x-1)**3).coeff(x**2)
```

```
[163]: 0
```



```
[165]: ((x-1)**3).expand().coeff(x**2)
```

```
[165]: -3
```

1.2.3 Racines et factorisation

```
[28]: P=x**3-2*x+1 ; Q=x**5+2*x**3-1 ; R=(x-1)**2*P ; display(P,Q,R)
```

$$x^3 - 2x + 1$$

$$x^5 + 2x^3 - 1$$

$$(x-1)^2 (x^3 - 2x + 1)$$

Les racines d'un polynôme sont calculées de façon exacte par la fonction `roots`. La réponse ne peut être complète que si le polynôme est de degré ≤ 4 ou qu'il peut être factorisé en produit de polynômes de degré ≤ 4 sur le domaine en cours (théoriquement une équation algébrique ne peut être résolue par radicaux que si elle est de degré ≤ 4). La sortie de `roots` est un dictionnaire formé de couples `racine:multiplicité`

```
[29]: roots(P)
```

```
[29]: {1: 1, -sqrt(5)/2 - 1/2: 1, -1/2 + sqrt(5)/2: 1}
```

```
[30]: roots(R)
```

```
[30]: {1: 3, -sqrt(5)/2 - 1/2: 1, -1/2 + sqrt(5)/2: 1}
```

```
[31]: roots(Q) # la réponse est l'ensemble vide car deg Q > 4 et Q est irréductible
      ↪ sur Q
```

```
[31]: {}
```

```
[32]: roots(R, multiple=true) # s'il est souhaitable d'avoir plutôt la liste des
      ↪ racines, chaque racine est répétée autant de fois que sa multiplicité dans
      ↪ la liste
```

```
[32]: [1, 1, 1, -1/2 + sqrt(5)/2, -sqrt(5)/2 - 1/2]
```

Pour la factorisation on dispose de la fonction généraliste `factor`. Elle peut être utilisée avec tout type d'expression. Avec des polynômes, la factorisation se fait sur le corps de base $\mathbb{Q}$. Il faudra l'aider en se plaçant dans un corps qui contient les racines pour obtenir une factorisation complète. Si elle est utilisée avec un `Poly` alors la réponse est un `'Poly'` et un `'Poly'` est toujours réécrit sous sa forme normale.

```
[33]: factor(P) # factorisant dans Q[X]
```

```
[33]: (x-1)(x^2+x-1)
```

On fournit le corps sur lequel doit être effectué la factorisation, ici $\mathbb{Q}[\sqrt{5}]$. L'option à utiliser est `extension`

```
[34]: factor(P,extension=sqrt(5)) #
```

```
[34]: 
$$(x-1) \left( x + \frac{1}{2} + \frac{\sqrt{5}}{2} \right) \left( x - \frac{\sqrt{5}}{2} + \frac{1}{2} \right)$$

```

Une option alléchante est prévue : `split`. Elle devrait «en principe» factoriser complètement le polynôme dans $\mathbb{C}[X]$ mais ne fait pour l'instant que lever une exception (une erreur). Il faudra peut être attendre l'avènement de l'informatique quantique :)

```
[35]: factor(x**3-2*x+1,split=True) # dans un futur proche :)
```

```
-----
NotImplementedError                                Traceback (most recent call last)
Cell In[35], line 1
----> 1 factor(x**3-2*x+1,split=True) # dans un futur proche :)

File ~/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/symp /
polys/polytools.py:6577, in factor(f, deep, *gens, **args)
    6574     return f.xreplace(partial)
    6576 try:
-> 6577     return _generic_factor(f, gens, args, method='factor')
    6578 except PolynomialError:
    6579     if not f.is_commutative:

File ~/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/symp /
polys/polytools.py:6256, in _generic_factor(expr, gens, args, method)
    6254 fraction = args.pop('fraction', True)
    6255 options.allowed_flags(args, [])
-> 6256 opt = options.build_options(gens, args)
    6257 opt['fraction'] = fraction
    6258 return _symbolic_factor(sympify(expr), opt, method)

File ~/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/symp /
polys/polyoptions.py:742, in build_options(gens, args)
    739     gens, args = (), gens
    741 if len(args) != 1 or 'opt' not in args or gens:
--> 742     return Options(gens, args)
    743 else:
    744     return args['opt']

File ~/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/symp /
polys/polyoptions.py:180, in Options.__init__(self, gens, args, flags, strict)
    177         raise OptionError("%s' option is not allowed together with,
-> '%s'" % (option, exclude_option))
    179 for option in self.__order__:
--> 180     self.__options__[option].postprocess(self)
```

```
File ~/.config/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/sympy/
↳ polys/polyoptions.py:519, in Split.postprocess(cls, options)
    516 @classmethod
    517 def postprocess(cls, options):
    518     if 'split' in options:
--> 519         raise NotImplementedError("'split' option is not implemented_
↳ yet")

NotImplementedError: 'split' option is not implemented yet
```

Au lieu de fournir explicitement l'extension à utiliser, on peut définir un alias sur une racine du polynôme et ensuite l'utiliser comme extension.

```
[36]: rac=AlgebraicNumber(RootOf(x**2+x-1,0),alias='omega') # rac revient à définir_
↳ le corps Q[omega], omega étant un alias de la première racine du polynôme_
↳ fournit à 'RootOf'
```

```
[37]: P_fact=factor(P,extension=rac) ; P_fact ## factorisation sur le corps Q[omega],_
↳ une sortie qui peut être beaucoup plus lisible
```

```
[37]: (x - 1) (x - ω) (x + ω + 1)
```

Regardons ce que cela donne si on développe cette expression (avec la méthode `expand`). En principe on devrait retrouver l'expression originale de P (sans ω)

```
[38]: P_fact.expand() # par très souple cette histoire d'extension, d'autre logiciels_
↳ font mieux, depuis longtemps
```

```
[38]: x3 - xω2 - x - xω + ω + ω2
```

En tant que simple expression, SymPy ne fait pas vraiment le lien avec la signification du symbole ω . Il faudra passer par un Polyet préciser que le calcul doit se faire dans le domaine $\mathbb{Q}[\omega]$.

```
[55]: P_fact.as_poly(extension=rac) # pour retrouver l'expression de P
```

```
[55]: Poly(x3 - 2x + 1, x, domain = QQ<omega>)
```

La fonction `factor_list` est plus spécialisée. Elle peut aussi agir sur des objets de type Poly et retourne la liste de ses facteurs irréductibles avec leurs évaluations. La factorisation s'effectue dans $\mathbb{Q}[X]$ à moins de préciser un autre corps d'extension (avec l'option `extension` par exemple).

```
[39]: P=(x-1)**3*(x-2)*(x**3-2*x+1)**2 ; P # on change de polynôme de test
```

```
[39]: (x - 2) (x - 1)3 (x3 - 2x + 1)2
```

```
[40]: factor_list(P,extension=rac)
```

```
[40]: (1, [(x - 2, 1), (x - 1, 2), (x - omega, 2), (x + omega + 1, 2), (x - 1, 3)])
```

```
[41]: factor_list(P.as_poly(),extension=rac) #liste des facteurs de dans l'anneau_
↳ rac[X]
```

```
[41]: (1,
      [(Poly(x - 2, x, domain='QQ<omega>'), 1),
       (Poly(x - omega, x, domain='QQ<omega>'), 2),
       (Poly(x + omega + 1, x, domain='QQ<omega>'), 2),
       (Poly(x - 1, x, domain='QQ<omega>'), 5)])
```

Essayons avec le polynôme Q (de degré 5 et irréductible sur $\mathbb{Q}$)

```
[42]: Q=poly(x**5+2*x**3-1) ; display(Q)
```

$\text{Poly}(x^5 + 2x^3 - 1, x, \text{domain} = \mathbb{Z})$

```
[43]: factor_list(Q)
```

```
[43]: (1, [(Poly(x**5 + 2*x**3 - 1, x, domain='ZZ'), 1)])
```

```
[44]: AA=AlgebraicNumber(RootOf(Q,2),alias='alpha') # la méthode utilisé avec ↵
      ↵ l'exemple précédent
```

```
[45]: factor_list(Q,extension=AA)
```

```
[45]: (1,
      [(Poly(x - alpha, x, domain='QQ<alpha>'), 1),
       (Poly(x**4 + alpha*x**3 + (2 + alpha**2)*x**2 + (alpha**3 + 2*alpha)*x +
        2*alpha**2 + alpha**4, x, domain='QQ<alpha>'),
        1)])
```

```
[46]: factor(Q.as_expr(),extension=AA)
```

```
[46]: (x - alpha) (x^4 + x^3 alpha + x^2 (2 + alpha^2) + x (alpha^3 + 2 alpha) + 2 alpha^2 + alpha^4)
```

Malheureusement les autres racines du polynôme Q ne s'expriment pas à l'aide de α . Sympy a bien sûr des solutions pour ce genre de situation mais on se contente des fonctions introduites jusqu'ici. Elles devraient couvrir nos besoins en algèbre linéaire.

1.2.4 Décomposition en éléments simples d'une fraction rationnelle

La fonction `apart` permet de décomposer une fraction rationnelle en éléments simples. La fraction peut être définie comme une simple expression. Les fonctions `together` et `cancel` permettent au contraire de rassembler une expression sous forme d'une fraction.

```
[139]: F=(x**2+1)/(x**3-1) ; G=1/(x-1)+x**2/(1-x**3)
      display(F,G)
```

$$\frac{x^2 + 1}{x^3 - 1}$$

$$\frac{x^2}{1 - x^3} + \frac{1}{x - 1}$$

[145]: `together(G) # ne fait que réécrire le fraction sous forme d'un quotient sans`
 $\hookrightarrow$ aucune transformation

[145]:
$$\frac{-x^3 + x^2(x-1) + 1}{(1-x^3)(x-1)}$$

[144]: `cancel(G) # procède en plus à une simplification des facteurs en commun.`

[144]:
$$\frac{x+1}{x^3-1}$$

[142]: `apart(F)`

[142]:
$$\frac{x-1}{3(x^2+x+1)} + \frac{2}{3(x-1)}$$

Comme pour `factor`, la factorisation du dénominateur se fait sur $\mathbb{Q}$. La fonction `apart` accepte toutes les options de `factor` et les utilise pour factoriser le dénominateur.

[158]: `apart(F, extension=(I, sqrt(3))) # les racines du dénominateurs sont dans le`
 $\hookrightarrow$ corps $\mathbb{Q}[I, \sqrt{3}]$

[158]:
$$-\frac{i(\sqrt{3}+i)}{3 \cdot (2x+1+\sqrt{3}i)} + \frac{i(\sqrt{3}-i)}{3 \cdot (2x+1-\sqrt{3}i)} + \frac{2}{3(x-1)}$$

Mais aussi de façon plus sophistiquée :

[153]: `JJ=AlgebraicNumber(RootOf(poly(x**2+x+1),0),alias='j') # on définit un alias 'j'`
 $\hookrightarrow$ sur la première racine de x^2+x+1

[152]: `apart(F, extension=JJ)`

[152]:
$$-\frac{j}{3(x+1+j)} + \frac{1+j}{3(x-j)} + \frac{2}{3(x-1)}$$

Une autre option est disponible : `full=True`. Elle déclenche un algorithme particulier pour les calculs.

[154]: `apart(F, full=True)`

[154]:
$$\text{RootSum} \left(w^2 + w + 1, \left(a \mapsto \frac{\frac{a}{3} + \frac{1}{3}}{-a+x} \right) \right) + \frac{2}{3(x-1)}$$

Pour forcer un calcul complet, on utilise la méthode `doit`

[160]: `apart(F, full=True).doit()`

[160]:
$$\frac{\frac{1}{6} - \frac{\sqrt{3}i}{6}}{x + \frac{1}{2} + \frac{\sqrt{3}i}{2}} + \frac{\frac{1}{6} + \frac{\sqrt{3}i}{6}}{x + \frac{1}{2} - \frac{\sqrt{3}i}{2}} + \frac{2}{3(x-1)}$$

2 Calcul matriciel avec SymPy

2.1 Définition de matrices

2.1.1 Constructeurs généralistes

`Matrix` permet de définir des matrices selon plusieurs variantes de syntaxe. `randMatrix` est une fonction très utile qui construit des matrices dont les coefficients sont choisis aléatoirement.

```
[362]: A=Matrix([[1,0,1],[2,1,-1],[3,-2,1]]) ; display(A) # liste de listes, chaque_
      ↪ liste est une ligne
```

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```
[363]: A=Matrix(((1,0,1),(2,1,-1),(3,-2,1))) ; display(A)
```

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

Utiliser des tuples est aussi accepté

La syntaxe pour construire un vecteur colonne est donc de la forme

```
[39]: Matrix([[1],[0],[-1]])
```

```
[39]: 
$$\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

```

Mais puisque l'utilisation des vecteurs colonnes est tellement courante, quand l'argument ne contient qu'une seule liste (et non une liste de listes) alors il est considéré comme donnant les coefficients d'une colonne

```
[40]: Matrix([1,0,-1])
```

```
[40]: 
$$\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

```

Cette facilité se généralise si on fournit à `Matrix` une liste de nom de vecteurs colonnes déjà définis. La matrice construite contient en une seule colonne les vecteurs empilés l'un sous l'autre

```
[49]: V1=Matrix([1,1,1])
      V2=Matrix([2,2,2])
      V3=Matrix([3,3,3])
      Matrix([V1,V2,V3])
```

```
[49]:
```

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 2 \\ 2 \\ 2 \\ 2 \\ 3 \\ 3 \\ 3 \end{bmatrix}$$

Une astuce permet de construire la matrice dont les vecteurs colonnes sont ceux dont les noms sont fournis dans la liste : modifier la taille de la matrice avec les méthode `reshape` et `T` (pour la transposée)

```
[48]: Matrix([V1,V2,V3]).reshape(3,3).T
```

```
[48]:  $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix}$ 
```

Cela ne fonctionne pas qu'avec des colonnes mais il vaut mieux dans ce cas utiliser les opérations de concaténation ou de construction par blocs pour un meilleur contrôle.

```
[79]: A1=Matrix(2,2,[1,1,1,1])
A2=Matrix(2,2,[2,2,2,2])
A3=Matrix(2,2,[3,3,3,3])
A4=Matrix(2,2,[4,4,4,4])
Matrix([A1,A2,A3,A4])
```

```
[79]:  $\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 2 & 2 \\ 2 & 2 \\ 3 & 3 \\ 3 & 3 \\ 4 & 4 \\ 4 & 4 \end{bmatrix}$ 
```

Il est possible de convertir une matrice en une liste Python. Il suffit d'utiliser pour cela la fonction `list`. C'est bien pratique pour récupérer la liste des coefficients d'une matrice pour les utiliser ensuite dans une équation par exemple.

```
[51]: list(A)
```

```
[51]: [1, 0, 1, 2, 1, -1, 3, -2, 1]
```

À ce propos, il ya une variante de syntaxe pour `Matrix` qui accepte dans ses arguments la liste de tous les coefficients à condition de préciser la taille de la matrice. Les coefficients sont alors lus ligne par ligne.

```
[52]: B=Matrix(3,3,[1,2,3,1,2,3,1,2,3]) ; display(B) # nombre de lignes, nombres de
      ↪ colonne et liste de tous les coefficients
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix}$$

```
[ ]: x,y,z=symbols('x y z')
M=Matrix(3,3, [1,1,1,x,y,z,x**2,y**2, z**2]) # symboles formels supporté à
      ↪ condition de les définir avec `symbols`
display(M)
```

$$\begin{bmatrix} 1 & 1 & 1 \\ x & y & z \\ x^2 & y^2 & z^2 \end{bmatrix}$$

```
[ ]: M.subs(z,10) # crée une nouvelle matrice avec z=10
```

```
[ ]:
```

$$\begin{bmatrix} 1 & 1 & 1 \\ x & y & 10 \\ x^2 & y^2 & 100 \end{bmatrix}$$

Il y a une autre syntaxe qui peut s'avérer très pratique dans beaucoup de cas : il est possible de fournir la taille de la matrice et une fonction qui définit les coefficients. Le constructeur Python `lambda` est accepté.

```
[53]: Matrix(9,9, lambda i,j : 1/(i+j+1)) # matrice de Cauchy de taille 9
```

```
[53]:
```

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} \\ \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} & \frac{1}{14} \\ \frac{1}{7} & \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} & \frac{1}{14} & \frac{1}{15} \\ \frac{1}{8} & \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} & \frac{1}{14} & \frac{1}{15} & \frac{1}{16} \\ \frac{1}{9} & \frac{1}{10} & \frac{1}{11} & \frac{1}{12} & \frac{1}{13} & \frac{1}{14} & \frac{1}{15} & \frac{1}{16} & \frac{1}{17} \end{bmatrix}$$

On peut aussi définir une fonction Python, ou bien un symbole qui désigne le nom d'une fonction, spécialement pour l'utiliser dans la définition d'une matrice.

```
[ ]: m = Function('m') # m est maintenant un symbole qui se comporte comme le nom
      ↪ d'une fonction sympy
def V (n) : # fonction python qui construit des matrices de Vandermonde
    return Matrix(n,n, lambda i,j : m(j)**i)
V(5)
```

```
[ ]:
```


$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ m(0) & m(1) & m(2) & m(3) & m(4) \\ m^2(0) & m^2(1) & m^2(2) & m^2(3) & m^2(4) \\ m^3(0) & m^3(1) & m^3(2) & m^3(3) & m^3(4) \\ m^4(0) & m^4(1) & m^4(2) & m^4(3) & m^4(4) \end{bmatrix}$$

Pour utiliser des valeurs particulières des abscisses on pourrait alors procéder à une substitution :

```
[ ]: V(5).subs({m(0):1, m(1):2, m(2):3, m(3):-1, m(4):5}) # subs sur un dictionnaire ↪
    ↪ de valeurs pour m
```

```
[ ]:
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & -1 & 5 \\ 1 & 4 & 9 & 1 & 25 \\ 1 & 8 & 27 & -1 & 125 \\ 1 & 16 & 81 & 1 & 625 \end{bmatrix}$$

Pour définir une matrice «circulante»

```
[73]: a=symbols('a:6')
Matrix(6,6, lambda i,j : a[(i+j) %6])
```

```
[73]:
```

$$\begin{bmatrix} a_0 & a_1 & a_2 & a_3 & a_4 & a_5 \\ a_1 & a_2 & a_3 & a_4 & a_5 & a_0 \\ a_2 & a_3 & a_4 & a_5 & a_0 & a_1 \\ a_3 & a_4 & a_5 & a_0 & a_1 & a_2 \\ a_4 & a_5 & a_0 & a_1 & a_2 & a_3 \\ a_5 & a_0 & a_1 & a_2 & a_3 & a_4 \end{bmatrix}$$

Rien n'empêche d'utiliser des instructions conditionnelles dans la fonction qui définit les coefficients

```
[194]: def f_indice (i,j) :
        if i<j or i==j :
            return a[i]
        else :
            return 0
Matrix(6,6, f_indice)
```

```
[194]:
```

$$\begin{bmatrix} a_0 & a_0 & a_0 & a_0 & a_0 & a_0 \\ 0 & a_1 & a_1 & a_1 & a_1 & a_1 \\ 0 & 0 & a_2 & a_2 & a_2 & a_2 \\ 0 & 0 & 0 & a_3 & a_3 & a_3 \\ 0 & 0 & 0 & 0 & a_4 & a_4 \\ 0 & 0 & 0 & 0 & 0 & a_5 \end{bmatrix}$$

Lors de la construction d'une matrice, SymPy ne fait que placer les coefficients selon l'expression donnée sans presque aucune transformation comme dans l'exemple suivant

```
[185]: Matrix(4,4, lambda i,j : (a[i]**2-a[j]**2)/(a[j]+a[i]))
```

```
[185]:
```

$$\begin{bmatrix} 0 & \frac{a_0^2-a_1^2}{a_0+a_1} & \frac{a_0^2-a_2^2}{a_0+a_2} & \frac{a_0^2-a_3^2}{a_0+a_3} \\ \frac{-a_0^2+a_1^2}{a_0+a_1} & 0 & \frac{a_1^2-a_2^2}{a_1+a_2} & \frac{a_1^2-a_3^2}{a_1+a_3} \\ \frac{-a_0^2+a_2^2}{a_0+a_2} & \frac{-a_1^2+a_2^2}{a_1+a_2} & 0 & \frac{a_2^2-a_3^2}{a_2+a_3} \\ \frac{-a_0^2+a_3^2}{a_0+a_3} & \frac{-a_1^2+a_3^2}{a_1+a_3} & \frac{-a_2^2+a_3^2}{a_2+a_3} & 0 \end{bmatrix}$$

Il faut en général faire une demande explicite de la transformation qu'on veut appliquer aux coefficients avec la méthode `applyfunc`

```
[187]: Matrix(4,4, lambda i,j : (a[i]**2-a[j]**2)/(a[j]+a[i])).applyfunc(simplify)
```

```
[187]:
```

$$\begin{bmatrix} 0 & a_0 - a_1 & a_0 - a_2 & a_0 - a_3 \\ -a_0 + a_1 & 0 & a_1 - a_2 & a_1 - a_3 \\ -a_0 + a_2 & -a_1 + a_2 & 0 & a_2 - a_3 \\ -a_0 + a_3 & -a_1 + a_3 & -a_2 + a_3 & 0 \end{bmatrix}$$

```
[193]: Matrix(4,4, lambda i,j : (a[i]**2-a[j]**2)/(a[j]+a[i])).applyfunc(simplify).
↪ applyfunc(lambda x:x**2)
```

```
[193]:
```

$$\begin{bmatrix} 0 & (a_0 - a_1)^2 & (a_0 - a_2)^2 & (a_0 - a_3)^2 \\ (-a_0 + a_1)^2 & 0 & (a_1 - a_2)^2 & (a_1 - a_3)^2 \\ (-a_0 + a_2)^2 & (-a_1 + a_2)^2 & 0 & (a_2 - a_3)^2 \\ (-a_0 + a_3)^2 & (-a_1 + a_3)^2 & (-a_2 + a_3)^2 & 0 \end{bmatrix}$$

Et pour clore cette balade, `randMatrix` permet de construire des matrices dont les coefficients sont générés aléatoirement. Elle prend un argument le nombre de lignes, celui des colonnes et encore deux arguments qui précisent l'intervalle dans lequel seraient pris les coefficients. D'autres syntaxes sont disponibles.

```
[174]: randMatrix(6,6,-5,5) # taille: (6,6), intervalle des coefficients : [-5,5]
```

```
[174]:
```

$$\begin{bmatrix} 4 & 1 & -3 & 0 & 5 & 4 \\ 0 & 5 & 1 & 2 & -3 & 1 \\ -1 & 0 & -3 & 2 & -4 & -3 \\ -4 & -4 & 5 & -1 & -5 & -4 \\ 3 & 0 & 4 & -4 & 5 & 4 \\ -4 & -1 & -4 & -2 & 3 & -2 \end{bmatrix}$$

Cette fonction est par exemple utile pour générer une matrice A à coefficients dans $\mathbb{Z}$ dont l'inverse est aussi à coefficients dans $\mathbb{Z}$, ce qui équivaut à $\det(A) = \pm 1$. Une telle matrice peut servir pour créer une matrice semblable à une matrice simple tout en conservant des coefficients dans $\mathbb{Z}$.

```
[178]: P=randMatrix(4,4,-2,2)
while P.det() != 1 :
    P=randMatrix(4,4,-2,2)
display(P, P.inv())
```

$$\begin{bmatrix} -2 & 1 & 2 & 2 \\ 1 & -1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 1 & 2 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & -1 & 0 \\ -1 & -4 & 6 & 2 \\ -1 & -4 & 7 & 2 \\ 2 & 7 & -11 & -3 \end{bmatrix}$$

2.1.2 Constructeurs spécialisés

Pas besoin d'explications pour les fonctions suivantes :

```
[ ]: eye(4)
```

```
[ ]:  $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ 
```

```
[ ]: zeros(3,4)
```

```
[ ]:  $\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ 
```

```
[ ]: ones(2,5)
```

```
[ ]:  $\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$ 
```

```
[ ]: diag(1,2,3,4) # construit une matrice diagonale
```

```
[ ]:  $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$ 
```

```
[ ]: diag(eye(3),Matrix(2,2,[2,1,0,2])) # par blocs, c'est aussi possible
```

```
[ ]:  $\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 0 & 2 \end{bmatrix}$ 
```

```
[74]: banded({0:(1,2,3)}) # généralisation de diag, l'argument doit être un  $\hookrightarrow$  dictionnaire Python
```

```
[74]:  $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$ 
```

```
[ ]: banded({0:(1,2,3,4), -1:(-1,-1,-1), 1:(1,1,1)}) # avec indexation des bandes à  $\hookrightarrow$  ajouter
```

```
[ ]:
```

$$\begin{bmatrix} 1 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 0 & -1 & 3 & 1 \\ 0 & 0 & -1 & 4 \end{bmatrix}$$

2.2 Manipulations de matrices

2.2.1 Matrices en tant qu'objets itérables

Un objet de type `Matrix` est itérable. Ce qui signifie qu'on peut, par exemple, faire une boucle sur les éléments qu'il contient.

```
[367]: A=Matrix([[1,0,1],[2,1,-1],[3,-2,1]]) ; display(A)
```

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```
[48]: for C in A : # A est itérable, cela peut avoir beaucoup d'intérêt
      print(f'Ceci est un coefficient de A : {C}')
```

```
Ceci est un coefficient de A : 1
Ceci est un coefficient de A : 0
Ceci est un coefficient de A : 1
Ceci est un coefficient de A : 2
Ceci est un coefficient de A : 1
Ceci est un coefficient de A : -1
Ceci est un coefficient de A : 3
Ceci est un coefficient de A : -2
Ceci est un coefficient de A : 1
```

On peut convertir une matrice `A` en un autre objet itérable, une liste par exemple, en récupérant la séquence de ces coefficients avec l'instruction `*A`. Celle-ci ne peut être utilisée que dans un contexte où Python s'attend à trouver une séquence d'éléments. L'instruction `'*Obj'` fonctionne avec tout objet `'Obj'` itérable.

L'instruction `list(A)` retourne aussi la liste des coefficients de `A`.

```
[368]: list(A)
```

```
[368]: [1, 0, 1, 2, 1, -1, 3, -2, 1]
```

```
[369]: *A
```

```
Cell In [369], line 1
    *A
    ^
SyntaxError: can't use starred expression here
```

```
[370]: [*A] # liste des coefficient de $A$, équivaleant à 'list(A)'
```

```
[370]: [1, 0, 1, 2, 1, -1, 3, -2, 1]
```

Cela marche dans le sens inverse : **Matrix** accepte qu'on lui fournisse une matrice à la place de la liste des coefficients (en fait tout objet itérable convient)

```
[372]: Matrix(1,9,A) # équivaleant à 'A.reshape(1,9)'
```

```
[372]: [1 0 1 2 1 -1 3 -2 1]
```

Une liste de listes formée chacune de matrices est aussi acceptée.

```
[377]: Matrix([[ones(2,2),zeros(2,3)], [zeros(3,2),ones(3,3)]])
```

```
[377]:
```

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Certaines des fonctions de transformations des expressions Sympy peuvent aussi agir sur des objets itérables, y compris les **Matrix**. La transformation est alors appliquée de chaque composant de l'objet. La fonction très utile **simplify** en est un exemple. Le même effet peut être obtenu avec la méthode **applyfunc** à laquelle on passe le nom d'une fonction le nom d'une fonction qui sera appliquée aux fonctions.

```
[28]: a=symbols('a:4')
A=Matrix(4,4, lambda i,j : (a[i]**2-a[j]**2)/(a[j]+a[i]))
B=Matrix(4,4, lambda i,j : (a[i]**2-a[j]**2)**2/(a[j]+a[i]))
display(A,B)
```

$$\begin{bmatrix} 0 & \frac{a_0^2-a_1^2}{a_0+a_1} & \frac{a_0^2-a_2^2}{a_0+a_2} & \frac{a_0^2-a_3^2}{a_0+a_3} \\ \frac{-a_0^2+a_1^2}{a_0+a_1} & 0 & \frac{a_1^2-a_2^2}{a_1+a_2} & \frac{a_1^2-a_3^2}{a_1+a_3} \\ \frac{-a_0^2+a_2^2}{a_0+a_2} & \frac{-a_1^2+a_2^2}{a_1+a_2} & 0 & \frac{a_2^2-a_3^2}{a_2+a_3} \\ \frac{-a_0^2+a_3^2}{a_0+a_3} & \frac{-a_1^2+a_3^2}{a_1+a_3} & \frac{-a_2^2+a_3^2}{a_2+a_3} & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & \frac{(a_0^2-a_1^2)^2}{a_0+a_1} & \frac{(a_0^2-a_2^2)^2}{a_0+a_2} & \frac{(a_0^2-a_3^2)^2}{a_0+a_3} \\ \frac{(-a_0^2+a_1^2)^2}{a_0+a_1} & 0 & \frac{(a_1^2-a_2^2)^2}{a_1+a_2} & \frac{(a_1^2-a_3^2)^2}{a_1+a_3} \\ \frac{(-a_0^2+a_2^2)^2}{a_0+a_2} & \frac{(-a_1^2+a_2^2)^2}{a_1+a_2} & 0 & \frac{(a_2^2-a_3^2)^2}{a_2+a_3} \\ \frac{(-a_0^2+a_3^2)^2}{a_0+a_3} & \frac{(-a_1^2+a_3^2)^2}{a_1+a_3} & \frac{(-a_2^2+a_3^2)^2}{a_2+a_3} & 0 \end{bmatrix}$$

```
[29]: simplify(A)
```

```
[29]:
```

$$\begin{bmatrix} 0 & a_0 - a_1 & a_0 - a_2 & a_0 - a_3 \\ -a_0 + a_1 & 0 & a_1 - a_2 & a_1 - a_3 \\ -a_0 + a_2 & -a_1 + a_2 & 0 & a_2 - a_3 \\ -a_0 + a_3 & -a_1 + a_3 & -a_2 + a_3 & 0 \end{bmatrix}$$

```
[30]: simplify(B) # pas vraiment la fonction qui convienne à la situation
```

```
[30]:
```

$$\begin{bmatrix} 0 & \frac{(a_0^2 - a_1^2)^2}{a_0 + a_1} & \frac{(a_0^2 - a_2^2)^2}{a_0 + a_2} & \frac{(a_0^2 - a_3^2)^2}{a_0 + a_3} \\ \frac{(a_0^2 - a_1^2)^2}{a_0 + a_1} & 0 & \frac{(a_1^2 - a_2^2)^2}{a_1 + a_2} & \frac{(a_1^2 - a_3^2)^2}{a_1 + a_3} \\ \frac{(a_0^2 - a_2^2)^2}{a_0 + a_2} & \frac{(a_1^2 - a_2^2)^2}{a_1 + a_2} & 0 & \frac{(a_2^2 - a_3^2)^2}{a_2 + a_3} \\ \frac{(a_0^2 - a_3^2)^2}{a_0 + a_3} & \frac{(a_1^2 - a_3^2)^2}{a_1 + a_3} & \frac{(a_2^2 - a_3^2)^2}{a_2 + a_3} & 0 \end{bmatrix}$$

```
[31]: cancel(B) # cancel est plus adéquate
```

```
[31]:
```

$$\begin{bmatrix} 0 & a_0^3 - a_0^2 a_1 - a_0 a_1^2 + a_1^3 & a_0^3 - a_0^2 a_2 - a_0 a_2^2 + a_2^3 & a_0^3 - a_0^2 a_3 - a_0 a_3^2 + a_3^3 \\ a_0^3 - a_0^2 a_1 - a_0 a_1^2 + a_1^3 & 0 & a_1^3 - a_1^2 a_2 - a_1 a_2^2 + a_2^3 & a_1^3 - a_1^2 a_3 - a_1 a_3^2 + a_3^3 \\ a_0^3 - a_0^2 a_2 - a_0 a_2^2 + a_2^3 & a_1^3 - a_1^2 a_2 - a_1 a_2^2 + a_2^3 & 0 & a_2^3 - a_2^2 a_3 - a_2 a_3^2 + a_3^3 \\ a_0^3 - a_0^2 a_3 - a_0 a_3^2 + a_3^3 & a_1^3 - a_1^2 a_3 - a_1 a_3^2 + a_3^3 & a_2^3 - a_2^2 a_3 - a_2 a_3^2 + a_3^3 & 0 \end{bmatrix}$$

```
[33]: factor(A) # factor ne prend pas en charge les itérables
```

```
[33]:
```

$$\begin{bmatrix} 0 & \frac{a_0^2 - a_1^2}{a_0 + a_1} & \frac{a_0^2 - a_2^2}{a_0 + a_2} & \frac{a_0^2 - a_3^2}{a_0 + a_3} \\ \frac{-a_0^2 + a_1^2}{a_0 + a_1} & 0 & \frac{a_1^2 - a_2^2}{a_1 + a_2} & \frac{a_1^2 - a_3^2}{a_1 + a_3} \\ \frac{-a_0^2 + a_2^2}{a_0 + a_2} & \frac{-a_1^2 + a_2^2}{a_1 + a_2} & 0 & \frac{a_2^2 - a_3^2}{a_2 + a_3} \\ \frac{-a_0^2 + a_3^2}{a_0 + a_3} & \frac{-a_1^2 + a_3^2}{a_1 + a_3} & \frac{-a_2^2 + a_3^2}{a_2 + a_3} & 0 \end{bmatrix}$$

```
[34]: A.applyfunc(factor)
```

```
[34]:
```

$$\begin{bmatrix} 0 & a_0 - a_1 & a_0 - a_2 & a_0 - a_3 \\ -a_0 + a_1 & 0 & a_1 - a_2 & a_1 - a_3 \\ -a_0 + a_2 & -a_1 + a_2 & 0 & a_2 - a_3 \\ -a_0 + a_3 & -a_1 + a_3 & -a_2 + a_3 & 0 \end{bmatrix}$$

```
[35]: B.applyfunc(factor)
```

```
[35]:
```

$$\begin{bmatrix} 0 & (a_0 - a_1)^2 (a_0 + a_1) & (a_0 - a_2)^2 (a_0 + a_2) & (a_0 - a_3)^2 (a_0 + a_3) \\ (a_0 - a_1)^2 (a_0 + a_1) & 0 & (a_1 - a_2)^2 (a_1 + a_2) & (a_1 - a_3)^2 (a_1 + a_3) \\ (a_0 - a_2)^2 (a_0 + a_2) & (a_1 - a_2)^2 (a_1 + a_2) & 0 & (a_2 - a_3)^2 (a_2 + a_3) \\ (a_0 - a_3)^2 (a_0 + a_3) & (a_1 - a_3)^2 (a_1 + a_3) & (a_2 - a_3)^2 (a_2 + a_3) & 0 \end{bmatrix}$$

2.2.2 Modification d'une matrice

Un objet de type `Matrix` est mutable. Tout changement de coefficient(s) se fait donc sur place. Certaines opérations créent toutefois une nouvelle matrice pour la sortie. La règle étant que si une opération ne retourne pas de résultat c'est que c'est la matrice sujet de l'opération qui a été modifiée.

On peut accéder aux coefficients d'une matrice, individuellement, par rangée (ligne ou colonne) ou par lot (en utilisant le slicing).

```
[53]: A[0,0]=2024 # modifie A
```

```
[54]: A
```

```
[54]: 
$$\begin{bmatrix} 2024 & 0 & 1 \\ 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```

```
[434]: A.row_del(0) # suppression de la 1ere ligne, modifie A. col_del est aussi disponible
```

```
[435]: A
```

```
[435]: 
$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```

```
[436]: A.col_join(Matrix(1,3,[1,2,3])) # la méthode col_join ajoute une ... ligne. Ne change pas A. row_join est aussi disponible.
```

```
[436]: 
$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

```

```
[437]: A # A n'a pas été modifiée
```

```
[437]: 
$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```

```
[438]: A[:,2] # slicing, ne modifie pas A
```

```
[438]: 
$$\begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

```

```
[439]: A[1,:]
```

```
[439]: 
$$\begin{bmatrix} 3 & -2 & 1 \end{bmatrix}$$

```

```
[440]: A[:,:] # toute la matrice, presque inutile ? pas vraiment
```

```
[440]: 
$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

```

On peut modifier toute une partie d'une matrice grâce au slicing. Cela donne une façon rapide pour faire des opérations élémentaires sur les lignes ou les colonnes de la matrice

```
[36]: A=randMatrix(3,3, 0, 3)
A
```

```
[36]: 
$$\begin{bmatrix} 2 & 2 & 2 \\ 0 & 1 & 3 \\ 2 & 0 & 0 \end{bmatrix}$$

```

```
[37]: A[0,:] *= 2 # multiplier la première ligne par 2
A
```

```
[37]:
```

$$\begin{bmatrix} 4 & 4 & 4 \\ 0 & 1 & 3 \\ 2 & 0 & 0 \end{bmatrix}$$

```
[38]: A[:,2] -= A[:,0] # retrancher la première colonne à la troisième.
A
```

```
[38]:
```

$$\begin{bmatrix} 4 & 4 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & -2 \end{bmatrix}$$

Le slicing produit une nouvelle matrice, y compris lorsque celle-ci ne contient qu'un seul coefficient (ou même aucun). On peut l'utiliser pour créer une copie d'une matrice (ou d'une partie de la matrice).

```
[441]: A[0:1,0:1] # c'est différent de A[0,0] car le slicing produit toujours un objet,
↳ de type 'Matrix'
```

```
[441]: [2]
```

```
[442]: A[3:3,:] # matrice vide, ne provoque pas d'erreur
```

```
[442]: ] [
```

Si on affecte à un nom un autre nom d'une matrice déjà créée, les deux noms pointent vers le même objet en mémoire. Opérer sur un nom implique donc que l'autre nom reflétera aussi les changements effectués.

```
[443]: A1=A # le nom A1 pointe vers la même matrice que A
A2=A[:,:] # crée une copie de $A$ qui est indépendante de $A$.
```

```
[445]: A1[1,1]=100 ; display(A,A1,A2) # on change A1, le changement est reflété sur A,
↳ les deux noms pointent vers le même contenu
```

$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & 100 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & 100 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & -1 \\ 3 & -2 & 1 \end{bmatrix}$$

2.2.3 Concaténation

```
[81]: A=randMatrix(3,3,1,5) ; B=randMatrix(2,3,-1,1) ; C=randMatrix(3,2,-1,1)
display(A,B,C)
```

$$\begin{bmatrix} 2 & 2 & 4 \\ 2 & 2 & 5 \\ 3 & 1 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ -1 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -1 \\ 0 & -1 \\ 1 & 1 \end{bmatrix}$$

`vstack` et `hstack` sont des méthodes attachées à l'objet `Matrix` qui permettent de concaténer respectivement verticalement ou horizontalement des matrices. Leur utilisation se fait soit sous la forme `Matrix.hstack` (comme s'il s'agissait d'une fonction) soit sous la forme `A.hstack` où `A` est le nom d'une matrice déjà créée.

```
[181]: Matrix.vstack(A,B)
```

```
[181]: 
$$\begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 3 \\ 4 & 5 & 3 \\ 0 & 0 & -1 \\ -1 & 1 & 1 \end{bmatrix}$$

```

```
[87]: Matrix.vstack(A,B,zeros(2,3))
```

```
[87]: 
$$\begin{bmatrix} 2 & 2 & 4 \\ 2 & 2 & 5 \\ 3 & 1 & 2 \\ 1 & 0 & -1 \\ -1 & 0 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

```

```
[83]: B.hstack(A,C) # pourtant 'B' n'a aucun rapport avec 'A' et 'C', l'important est_
      ↪ que c'est un objet 'Matrix'
```

```
[83]: 
$$\begin{bmatrix} 2 & 2 & 4 & 1 & -1 \\ 2 & 2 & 5 & 0 & -1 \\ 3 & 1 & 2 & 1 & 1 \end{bmatrix}$$

```

2.2.4 Methodes et propriétés de l'objet `Matrix`

```
[276]: A=randMatrix(4,4,-3,3) ; U=ones(4,4) ; display(A,U)
```

$$\begin{bmatrix} 2 & -2 & 1 & -1 \\ -3 & -3 & 2 & 0 \\ 3 & 0 & 1 & 2 \\ 0 & 2 & 3 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

```
[277]: A.rows # nombre de ligne
```

```
[277]: 4
```

```
[278]: A.cols # nombre de colonnes. C'est une propriété de l'objet et non une méthode
      ↪ (pas de parenthèses)
```

```
[278]: 4
```

```
[279]: A.shape # taille de la matrice
```

```
[279]: (4, 4)
```

```
[280]: A.det() # déterminant de A
```

```
[280]: 145
```

```
[281]: A.rank() # rang de A
```

```
[281]: 4
```

```
[282]: A.trace() # trace de A
```

```
[282]: 0
```

```
[283]: A.T # transposée de A
```

```
[283]: 
$$\begin{bmatrix} 2 & -3 & 3 & 0 \\ -2 & -3 & 0 & 2 \\ 1 & 2 & 1 & 3 \\ -1 & 0 & 2 & 0 \end{bmatrix}$$

```

```
[284]: A.inv() # inverse de A
```

```
[284]: 
$$\begin{bmatrix} \frac{26}{145} & -\frac{18}{145} & \frac{13}{145} & -\frac{1}{145} \\ -\frac{18}{145} & -\frac{21}{145} & -\frac{9}{145} & \frac{23}{145} \\ \frac{12}{145} & \frac{14}{145} & \frac{6}{145} & \frac{33}{145} \\ -\frac{9}{29} & \frac{4}{29} & \frac{10}{29} & -\frac{3}{29} \end{bmatrix}$$

```

```
[285]: A.cofactorMatrix() # comatrice de A
```

```
[285]: 
$$\begin{bmatrix} 26 & -18 & 12 & -45 \\ -18 & -21 & 14 & 20 \\ 13 & -9 & 6 & 50 \\ -1 & 23 & 33 & -15 \end{bmatrix}$$

```

```
[ ]: A.adjugate() # matrice conjointe de A, c'est la transposée de sa comatrice
```

```
[ ]: 
$$\begin{bmatrix} 26 & -18 & 13 & -1 \\ -18 & -21 & -9 & 23 \\ 12 & 14 & 6 & 33 \\ -45 & 20 & 50 & -15 \end{bmatrix}$$

```

```
[291]: A*A.adjugate() == A.det()*eye(4) # vérification d'une certaine formule fameuse
```

```
[291]: True
```

```
[218]: U.columnspace() # une base extraite des colonnes de A
```

```
[218]: Matrix([
      [1],
      [1],
      [1],
      [1]])
```

```
[219]: U.rowspace() # une base extraite des lignes de A
```

```
[219]: Matrix([[1, 1, 1, 1]])
```

```
[220]: U.nullspace() # une base du noyau de A
```

```
[220]: Matrix([
      [-1],
      [ 1],
      [ 0],
      [ 0]]),
Matrix([
      [-1],
      [ 0],
      [ 1],
      [ 0]]),
Matrix([
      [-1],
      [ 0],
      [ 0],
      [ 1]])
```

Pour former la matrice dont les colonnes sont formées par la base retournée par `nullspace` on pourrait utiliser une instruction de la forme

```
[221]: V1,V2,V3=U.nullspace() ; Matrix.hstack(V1,V2,V3) # former la matrice dont les
      ↪ colonnes forment une base de ker U
```

```
[221]: 
$$\begin{bmatrix} -1 & -1 & -1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

```

on peut aussi utiliser directement une «comprehension» Python pour former la liste des vecteurs et «resaper» adéquatement la matrice

```
[27]: Matrix([V for V in U.nullspace()]).reshape(4,3)
```

```
[27]:
```

$$\begin{bmatrix} -1 & 1 & 0 \\ 0 & -1 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

La matrice résultat de la méthode de Gauss-Jordan peut être calculée avec la méthode `rref`. Celle-ci retourne un tuple, la matrice réduite elle-même et un tuple formé des numéros des colonnes sur lesquelles les pivots ont été pris.

```
[222]: A.rref() # reduite
```

```
[222]: (Matrix([
  [1, 0, 0, 0],
  [0, 1, 0, 0],
  [0, 0, 1, 0],
  [0, 0, 0, 1]]),
  (0, 1, 2, 3))
```

```
[277]: A.charpoly() # polynôme caractéristique de A
```

```
[277]: PurePoly( $\lambda^4 - 4\lambda^3 + 8\lambda^2 + 2\lambda + 7$ ,  $\lambda$ , domain =  $\mathbb{Z}$ )
```

```
[278]: A.charpoly(x) # pour changer de symbole pour l'indéterminée
```

```
[278]: PurePoly( $x^4 - 4x^3 + 8x^2 + 2x + 7$ ,  $x$ , domain =  $\mathbb{Z}$ )
```

```
[279]: U.charpoly(x)
```

```
[279]: PurePoly( $x^4 - 4x^3$ ,  $x$ , domain =  $\mathbb{Z}$ )
```

Les méthodes `eigenvals` et `eigenvecs` permettent d'obtenir les informations sur les éléments propres de l'objet `Matrix`. Tous les calculs sont effectués de manière algébrique (exacte). + `eigenvals` fournit les valeurs propres de la matrice sous forme d'un dictionnaire 'valeur propre:multiplicité'. Avec l'option `multiple=True` la réponse est une liste où chaque valeur propre est répétée autant de fois que sa multiplicité. + `eigenvecs` fournit une liste de tuples, chaque tuple est formé d'une valeur propre, de sa multiplicité et d'une liste de vecteurs qui forment une base du sous-espace propre associé. Assez d'informations donc pour savoir si une matrice est diagonalisable et le cas échéant de former une matrice de passage dans une base de diagonalisation (moyennant quelques manipulations). + Il y a des méthodes capables de réduire directement une matrice en fournissant en réponse une matrice de passage et une matrice réduite (voir section réduction ...). Mais elles peuvent être très lentes si les valeurs propres de la matrice sont compliquées. Chose dont ne souffre pas `eigenvecs`.

```
[280]: A.eigenvals() # valeurs propres de A, les valeurs sont calculées de façon
↪ exacte.
```

```
[280]: {1 + sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3)) + 20/sqrt(-4/3 +
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))
- 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2 - sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3)))/2: 1,
```

```

1 - sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3))/2 - sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) +
20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2: 1,
1 + sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) - 20/sqrt(-4/3 +
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))
- 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2 + sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/2: 1,
1 + sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3))/2 - sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) -
20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2: 1}

```

```
[281]: U.eigenvals() # la sortie est un dictionnaire de vap:mult
```

```
[281]: {4: 1, 0: 3}
```

```
[282]: U.eigenvals(multiple=True) # pour avoir une réponse sous forme d'une liste
```

```
[282]: [0, 0, 0, 4]
```

```
[284]: A.eigenvects() # liste de tuples, chaque tuple est formé d'une vap, sa
↳ multiplicité et une liste de vecteurs formant une base du sep
```

```
[284]: [(1 - sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3))/2 - sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) +
20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2,
1,
[Matrix([
[ 16/27 + 2*(1 - sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) +
2*(175/108 + sqrt(1703)*I/4)**(1/3))/2 - sqrt(-8/3 - 2*(175/108 +
sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3))
+ 2*(175/108 + sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)))/2]**3/27 - 4*sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/27 - 7*(1 -
sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3))/2 - sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) +
20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2]**2/27 -
4*sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 +
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))
- 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/27],
[ -14/27 + 2*sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) +
20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/27 + 2*(1 -
sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +

```



```

sqrt(1703)*I/4)**(1/3))/2)**3/27 + 2*sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/27 + 2*(1 +
sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 + 86/(9*(175/108
+ sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3)) -
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/2 - sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/2)**2/27 -
2*sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 +
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))
- 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/27],
[101/81 + 43*sqrt(-8/3 - 2*(175/108 + sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 +
86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))
- 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)))/162 - 17*(1 + sqrt(-8/3 - 2*(175/108
+ sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108
+ sqrt(1703)*I/4)**(1/3)))/2 - sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/2)**2/81 -
43*sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3)) + 2*(175/108 +
sqrt(1703)*I/4)**(1/3))/162 + 4*(1 + sqrt(-8/3 - 2*(175/108 +
sqrt(1703)*I/4)**(1/3) + 20/sqrt(-4/3 + 86/(9*(175/108 + sqrt(1703)*I/4)**(1/3))
+ 2*(175/108 + sqrt(1703)*I/4)**(1/3)) - 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)))/2 - sqrt(-4/3 + 86/(9*(175/108 +
sqrt(1703)*I/4)**(1/3)) + 2*(175/108 + sqrt(1703)*I/4)**(1/3))/2)**3/81],
[
1]]))]]

```

```
[252]: U.eigenvects()
```

```

[252]: [(0,
3,
[Matrix([
[-1],
[ 1],
[ 0],
[ 0]]),
Matrix([
[-1],
[ 0],
[ 1],
[ 0]]),
Matrix([
[-1],
[ 0],
[ 0],
[ 1]])]),
(4,
1,
[Matrix([

```

```
[1],
[1],
[1],
[1]]]]]
```

2.2.5 Nom symbolique de matrices

Comme pour `symbols`, il est possible de déclarer un nom comme désignant symboliquement une matrice avec la fonction `MatrixSymbol`. Bien que les possibilités soient plus limitées, ses matrices peuvent être utilisées dans des expressions Sympy.

```
[266]: A=MatrixSymbol('A',3,3)
      B=MatrixSymbol('B',3,3)
```

Dans une expression qui contient le nom d'une matrice symbolique, celui-ci ne sera pas développée en son contenu (contrairement à une matrice définie par `Matrix`). Pour afficher le contenu d'une expression qui contient des matrices symboliques, utiliser la méthode `as_explicit`.

```
[268]: A.as_explicit()
```

```
[268]: 
$$\begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{bmatrix}$$

```

```
[277]: (A-eye(3))**2
```

```
[277]: 
$$\left( \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix} + A \right)^2$$

```

```
[280]: ((A-eye(3))**2).as_explicit()
```

```
[280]: 
$$\begin{bmatrix} (A_{0,0}-1)^2 + A_{0,1}A_{1,0} + A_{0,2}A_{2,0} & (A_{0,0}-1)A_{0,1} + (A_{1,1}-1)A_{0,1} + A_{0,2}A_{2,1} & (A_{0,0}-1)A_{0,2} + (A_{1,1}-1)A_{0,2} + A_{0,2}A_{2,2} \\ (A_{0,0}-1)A_{1,0} + (A_{1,1}-1)A_{1,0} + A_{1,2}A_{2,0} & (A_{1,1}-1)^2 + A_{0,1}A_{1,0} + A_{1,2}A_{2,1} & (A_{1,1}-1)A_{1,2} + (A_{2,2}-1)A_{1,2} + A_{1,2}A_{2,2} \\ (A_{0,0}-1)A_{2,0} + (A_{2,2}-1)A_{2,0} + A_{1,0}A_{2,1} & (A_{1,1}-1)A_{2,1} + (A_{2,2}-1)A_{2,1} + A_{0,1}A_{2,0} & (A_{2,2}-1)^2 + A_{2,2}A_{2,2} \end{bmatrix}$$

```

Les fonctions `Identity` et `ZeroMatrix` permettent de construire des matrices identités et des matrices nulles de la taille désirée et qui se comportent comme des noms symboliques

```
[281]: J=Identity(3)
      Z=ZeroMatrix(3,3)
      O=OneMatrix(3,3)
      display(J,Z,O)
```

```
⌈
```

```
0
```

```
1
```

```
[274]: Z*A
```

```
[274]: 0
```

```
[271]: J*A
```

```
[271]: A
```

```
[285]: (2*I+J).as_explicit()
```

```
[285]: 
$$\begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix}$$

```

```
[ ]: (A*B).inv()
```

$B^{-1}A^{-1}$

```
[ ]: (A*B).T
```

$B^T A^T$

```
[ ]: det(A*B)
```

$|A| |B|$

```
[ ]: trace(A+B)
```

$\text{tr}(A) + \text{tr}(B)$

Malheureusement les fonctionnalités disponibles pour ce type de construction restent limitées.

```
[ ]: ((A+B)**2).expand() # on aimerait bien avoir une forme développée en respectant la non commutativité.
```

$(A+B)^2$

2.2.6 Matrices par blocs

La fonction `BlockMatrix` permet de créer des objets du même nom et qui ont un type à part entière. Les méthodes de l'objet `Matrix` ne sont pas toutes utilisables avec les objets `BlockMatrix`.

`BlockMatrix` attend une liste de listes, chaque liste étant formée de nom de matrices ou de matrices explicites.

```
[238]: A=zeros(2,2)
      B=ones(2,3)
      C=ones(3,2)
      D=zeros(3,3)
      M=BlockMatrix([[A,B],[C,D]])
      display(M)
```

$$\begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} & \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix}$$

```
[239]: M.shape
```

```
[239]: (5, 5)
```

```
[240]: M**2
```

```
[240]:
```

$$\left(\begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} & \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix} \right)^2$$

Les opérations sur les `BlockMatrix` ne sont pas évaluées jusqu'au bout vu la nature symbolique de ces objets. Il faut demander explicitement une évaluation avec la fonction `bloc_collapse`

```
[241]: block_collapse(M**2+eye(5))
```

```
[241]:
```

$$\begin{bmatrix} \mathbb{I} + \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} & \mathbb{I} + \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix} \end{bmatrix}$$

```
[242]: (M**2+eye(5)).as_explicit()
```

```
[242]:
```

$$\begin{bmatrix} 4 & 3 & 0 & 0 & 0 \\ 3 & 4 & 0 & 0 & 0 \\ 0 & 0 & 3 & 2 & 2 \\ 0 & 0 & 2 & 3 & 2 \\ 0 & 0 & 2 & 2 & 3 \end{bmatrix}$$

`BlockMatrix` exige que les blocs soient découpés correctement et renvoie vers une utilisation de `Matrix` si cette condition n'est pas remplie

```
[243]: BlockMatrix([[A,C],[B,D]])
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In [243], line 1
----> 1 BlockMatrix([[A,C],[B,D]])

File /usr/lib/python3/dist-packages/sympy/matrices/expressions/blockmatrix.py:
  122, in BlockMatrix.__new__(cls, *args, **kwargs)
    115         if blocky and ok:
    116             raise ValueError(filldedent('''
```

```

117         Although this matrix is comprised of blocks,
118         the blocks do not fill the matrix in a
119         size-symmetric fashion. To create a full matrix
120         from these arguments, pass them directly to
121         Matrix.'')
--> 122     raise ValueError(filldedent('''
123         When there are not the same number of rows in each
124         row's matrices or there are not the same number of
125         total columns in each row, the matrix is not a
126         block matrix. If this matrix is known to consist of
127         blocks fully filling a 2-D space then see
128         Matrix.irregular.''))
129 mat = ImmutableDenseMatrix(rows, evaluate=False)
130 obj = Basic.__new__(cls, mat)

```

ValueError:

When there are not the same number of rows in each row's matrices or there are not the same number of total columns in each row, the matrix is not a block matrix. If this matrix is known to consist of blocks fully filling a 2-D space then see `Matrix.irregular`.

Il est possible de définir des noms symboliques pour des matrices et ensuite de les utiliser dans des `BlockMatrix`

```

[244]: A=MatrixSymbol('A',2,2)
      B=MatrixSymbol('B',2,3)
      C=MatrixSymbol('C',3,2)
      D=MatrixSymbol('D',3,3)
      M=BlockMatrix([[A,B],[C,D]])
      display(M)

```

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

La méthode `as_explicit` permet d'afficher le contenu détaillé de la matrice

```

[245]: M.as_explicit()

```

```

[245]: 
$$\begin{bmatrix} A_{0,0} & A_{0,1} & B_{0,0} & B_{0,1} & B_{0,2} \\ A_{1,0} & A_{1,1} & B_{1,0} & B_{1,1} & B_{1,2} \\ C_{0,0} & C_{0,1} & D_{0,0} & D_{0,1} & D_{0,2} \\ C_{1,0} & C_{1,1} & D_{1,0} & D_{1,1} & D_{1,2} \\ C_{2,0} & C_{2,1} & D_{2,0} & D_{2,1} & D_{2,2} \end{bmatrix}$$


```

L'indexation fera référence au contenu de la matrice coefficient pas coefficient.

```

[246]: M[3,4]

```

```

[246]:  $D_{1,2}$ 

```

[247]: `M**2`

[247]:
$$\left(\begin{bmatrix} A & B \\ C & D \end{bmatrix} \right)^2$$

[248]: `block_collapse(M**2)`

[248]:
$$\begin{bmatrix} A^2 + BC & AB + BD \\ CA + DC & D^2 + CB \end{bmatrix}$$

[249]: `(M**2).as_explicit()`

[249]:
$$\begin{bmatrix} A_{0,0}^2 + A_{0,1}A_{1,0} + B_{0,0}C_{0,0} + B_{0,1}C_{1,0} + B_{0,2}C_{2,0} & A_{0,0}A_{0,1} + A_{0,1}A_{1,1} + B_{0,0}C_{0,1} + B_{0,1}C_{1,1} + B_{0,2}C_{2,1} & A_{0,0}A_{1,0} + A_{1,0}A_{1,1} + B_{1,0}C_{0,0} + B_{1,1}C_{1,0} + B_{1,2}C_{2,0} & A_{0,1}A_{1,0} + A_{1,1}^2 + B_{1,0}C_{0,1} + B_{1,1}C_{1,1} + B_{1,2}C_{2,1} & A_{0,0}C_{0,0} + A_{1,0}C_{0,1} + C_{0,0}D_{0,0} + C_{1,0}D_{0,1} + C_{2,0}D_{0,2} & A_{0,1}C_{0,0} + A_{1,1}C_{0,1} + C_{0,1}D_{0,0} + C_{1,1}D_{0,1} + C_{2,1}D_{0,2} & A_{0,0}C_{1,0} + A_{1,0}C_{1,1} + C_{0,0}D_{1,0} + C_{1,0}D_{1,1} + C_{2,0}D_{1,2} & A_{0,1}C_{1,0} + A_{1,1}C_{1,1} + C_{0,1}D_{1,0} + C_{1,1}D_{1,1} + C_{2,1}D_{1,2} & A_{0,0}C_{2,0} + A_{1,0}C_{2,1} + C_{0,0}D_{2,0} + C_{1,0}D_{2,1} + C_{2,0}D_{2,2} & A_{0,1}C_{2,0} + A_{1,1}C_{2,1} + C_{0,1}D_{2,0} + C_{1,1}D_{2,1} + C_{2,1}D_{2,2} \end{bmatrix}$$

Le calcul de l'inverse est possible avec les objets **BlockMatrix**. SymPy ne se soucie par contre pas des conditions de validité de celui-ci

[250]: `block_collapse(M**(-1))`

[250]:
$$\begin{bmatrix} A^{-1} + A^{-1}B(-CA^{-1}B + D)^{-1}CA^{-1} & -A^{-1}B(-CA^{-1}B + D)^{-1} \\ -(-CA^{-1}B + D)^{-1}CA^{-1} & (-CA^{-1}B + D)^{-1} \end{bmatrix}$$

[251]: `J=BlockDiagMatrix(Identity(2),Identity(3)) ; J`

[251]:
$$\begin{bmatrix} \mathbb{I} & 0 \\ 0 & \mathbb{I} \end{bmatrix}$$

[252]: `block_collapse(M-x*J)`

[252]:
$$\begin{bmatrix} -x\mathbb{I} + A & B \\ C & -x\mathbb{I} + D \end{bmatrix}$$

La transposée est disponible à travers la méthode **transpose** (T n'est pas accepté)

[253]: `M.transpose()`

[253]:
$$\begin{bmatrix} A^T & C^T \\ B^T & D^T \end{bmatrix}$$

[254]: `M.det()`

[254]:
$$\begin{vmatrix} A & B \\ C & D \end{vmatrix}$$

[273]: `M_tri=BlockMatrix([[A,B],[ZeroMatrix(3,2),D]])
display(M_tri)`

[273]:
$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix}$$

```
[ ]: M_tri.det() # règle de calcul non implémentée pour une matrice triangulaire
      ↪ par blocs
```

```
[ ]:  $\begin{vmatrix} A & B \\ 0 & D \end{vmatrix}$ 
```

```
[ ]: M_tri.charpoly() # même pas de méthode 'charpoly'.
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In [275], line 1
----> 1 M_tri.charpoly()

AttributeError: 'BlockMatrix' object has no attribute 'charpoly'
```

2.3 Résolution d'équations

Sympy dispose de toute une panoplie de résolveurs d'équations. Ceux qui pourront nous concerner ici sont : + **solve** : fonction généraliste qui permet de résoudre tout type d'équation, ou système d'équations. Bien que fournissant différents formats pour les solutions selon le type d'équations, la sortie est en général relativement simples à récupérer pour les calculs ultérieurs. + **solveset** : une version plus récente plus rigoureuse et avec une interface plus homogène entre les différents type d'équations qui s'efforce de trouver toutes les solutions. La sortie est difficile à exploiter. + **linsolve** : fonction spécialisée dans la résolution de systèmes linéaires. Elle s'appuie sur **solveset** **linsolve** semble indiquée pour les problèmes de l'algèbre linéaire, on lui préférera toutefois la fonction **solve** pour sa souplesse. Le fait que celle-ci retourne un dictionnaire en général, permet de récupérer les solutions par un simple appel à la méthode **subs**

```
[42]: x,y,z,t=symbols('x y z t')
      a,b,c,d=symbols('a b c d')
```

solve accepte en entrée + une expression 'expr' ou un objet itérable d'expressions (liste, tuple, tableau ou même matrice), par exemple ['expr1', 'expr2', ...]. L'équation à résoudre est alors 'expr=0' ou [expr1=0, expr2=0, ...] + une équation formaté avec le constructeur **Eq** sous la forme **Eq(expr, b)** ou 'E' est une expression contenant une ou plusieurs inconnue et 'b' est le second membre. L'équation à résoudre est alors 'expr=b'. + sans aucune autre indication, tout symbole utilisé dans le(s) équation(s) est considéré comme inconnu. + on peut préciser les inconnues sous forme d'une séquence non encadrée de symboles ou sous forme d'un objet itérable contenant les noms des inconnues. Dans ce cas les éventuels symboles dans l'équation qui ne sont pas des inconnues sont considérés comme des paramètres.

La sortie est en général un dictionnaire qui représente les solutions trouvés. Elle peut être vide bien que l'équation admet des solutions (dans certains cas vraiment évidentes).

```
[43]: solve([a*x+b*y-1, c*x+d*y],x,y) # les inconnues sont x et y. a et b sont des
      ↪ paramètres
```

```
[43]: {x: d/(a*d - b*c), y: -c/(a*d - b*c)}
```

```
[44]: solve([a*x+b*y-1, c*x+d*y],(x,y))
```

```
[44]: {x: d/(a*d - b*c), y: -c/(a*d - b*c)}
```

```
[45]: solve((a*x+b*y-1, c*x+d*y),[x,y])
```

```
[45]: {x: d/(a*d - b*c), y: -c/(a*d - b*c)}
```

Créons maintenant des objets de type matriciel que nous utiliserons dans les exemples suivants

```
[46]: V1=Matrix([1,1,0,-1])
      V2=Matrix([-1,0,0,2])
      A=Matrix([V1,V2,V1-V2,V1+V2]).reshape(4,4) # matrice de rang 2
      X=Matrix(4,1,[x,y,z,t]) # un vecteur colonne à coefficients inconnus
      M=MatrixSymbol('x',4,4).as_explicit() # une matrice carrée à coefficients
      ↪ inconnus
```

```
[47]: M
```

```
[47]: 
$$\begin{bmatrix} x_{0,0} & x_{0,1} & x_{0,2} & x_{0,3} \\ x_{1,0} & x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,0} & x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,0} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

```

```
[48]: solX=solve(A*X) ; solX
```

```
[48]: {t: -y, x: -2*y}
```

et on peut ainsi récupérer les solutions avec un `subs` sur le vecteur 'X'

```
[49]: X.subs(solX)
```

```
[49]: 
$$\begin{bmatrix} -2y \\ y \\ z \\ -y \end{bmatrix}$$

```

et même une base de l'ensemble des solutions avec

```
[50]: X1=X.subs(solX).subs({y:1,z:0})
      X2=X.subs(solX).subs({y:0,z:1})
      display(X1,X2)
```

```

$$\begin{bmatrix} -2 \\ 1 \\ 0 \\ -1 \end{bmatrix}$$

```

```

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

```


Dans la suite on veut déterminer toutes les matrices qui commutent avec V

```
[52]: solM=solve(A*M-M*A,M)
```

```
[54]: solM
```

```
[54]: {x[0, 0]: 3*x[3, 0] + x[3, 2] + x[3, 3],
      x[0, 1]: x[3, 0] + x[3, 1] + 2*x[3, 2],
      x[0, 2]: 2*x[3, 2],
      x[0, 3]: -x[3, 0] - x[3, 1],
      x[1, 0]: -x[3, 1] + 2*x[3, 2],
      x[1, 1]: x[3, 0] - x[3, 1] + x[3, 2] + x[3, 3],
      x[1, 2]: -x[3, 2],
      x[1, 3]: -x[3, 0] + 2*x[3, 1] - 3*x[3, 2],
      x[2, 0]: -x[2, 2] - x[2, 3] + 3*x[3, 0] - 2*x[3, 1] + 5*x[3, 2] + x[3, 3],
      x[2, 1]: x[2, 2] - x[2, 3] - x[3, 1] + x[3, 2] - x[3, 3]}
```

et on n'a qu'à injecter les solutions dans la matrice M avec un `subs`

```
[55]: M.subs(solM)
```

```
[55]:
```

$$\begin{bmatrix} 3x_{3,0} + x_{3,2} + x_{3,3} & x_{3,0} + x_{3,1} + 2x_{3,2} & 2x_{3,2} & -x_{3,0} - x_{3,1} \\ -x_{3,1} + 2x_{3,2} & x_{3,0} - x_{3,1} + x_{3,2} + x_{3,3} & -x_{3,2} & -x_{3,0} + 2x_{3,1} - 3x_{3,2} \\ -x_{2,2} - x_{2,3} + 3x_{3,0} - 2x_{3,1} + 5x_{3,2} + x_{3,3} & x_{2,2} - x_{2,3} - x_{3,1} + x_{3,2} - x_{3,3} & x_{2,2} & x_{2,3} \\ x_{3,0} & x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

et cette fois former une base de $\mathcal{C}(V)$ par une selection à travers la méthode `applyfunc`

```
[56]: M.subs(solM).applyfunc(lambda z:z.coef(M[3,0]))
```

```
[56]:
```

$$\begin{bmatrix} 3 & 1 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ 3 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

```
[57]: for (i,j) in ((3,0),(3,1),(3,2),(3,3),(2,0),(2,1)) :
      display(M.subs(solM).applyfunc(lambda z:z.coef(M[i,j])))
```

$$\begin{bmatrix} 3 & 1 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ 3 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 & -1 \\ -1 & -1 & 0 & 2 \\ -2 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 2 & 0 \\ 2 & 1 & -1 & -3 \\ 5 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Réduction d'une matrice

La méthode `diagonalize` permet de diagonaliser une matrice. Elle retourne une erreur si la matrice n'est pas diagonalisable. Toutefois si les valeurs propres de la matrice sont compliquées alors la réponse peut prendre beaucoup de temps. Ce qui ne se produit pas quand on demande directement les éléments propres de la matrice avec `eigenvecs`. À croire que ce qui est responsable du ralentissement n'est pas tellement les calculs mais toutes les phases de test et de simplification par lesquelles doit normalement passer l'algorithme pour couvrir tous les cas possibles. Une fonction python est proposée dans l'exemple 3 à la fin de cette planche et se basant sur `eigenvecs`.

[285]: `A.diagonalize()` # contrairement à 'eigenvecs' qui fournit une réponse rapide, `diagonalize` peut prendre beaucoup de temps avant de fournir une réponse.
 ↳ Calcul interrompu après 5min.

```
-----
KeyError                                Traceback (most recent call last)
File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:479, in
↳ make_property.<locals>.getit(self)
    478 try:
--> 479     return self._assumptions[fact]
    480 except KeyError:
```

KeyError: 'infinite'

During handling of the above exception, another exception occurred:

```
KeyError                                Traceback (most recent call last)
File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:479, in
↳ make_property.<locals>.getit(self)
    478 try:
--> 479     return self._assumptions[fact]
    480 except KeyError:
```

KeyError: 'integer'

During handling of the above exception, another exception occurred:

```
KeyboardInterrupt                                Traceback (most recent call last)
Cell In [285], line 1
----> 1 A.diagonalize() # contrairement à 'eigenvects' qui fournit une réponse
↳ rapide, diagonalize peut prendre beaucoup de temps avant de fournir une
↳ réponse. Calcul interrompu après 5min.

File /usr/lib/python3/dist-packages/sympy/matrices/matrices.py:388, in
↳ MatrixEigen.diagonalize(self, reals_only, sort, normalize)
    387 def diagonalize(self, reals_only=False, sort=False, normalize=False):
--> 388     return _diagonalize(self, reals_only=reals_only, sort=sort,
    389                        normalize=normalize)

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:717, in
↳ _diagonalize(M, reals_only, sort, normalize)
    714 if not M.is_square:
    715     raise NonSquareMatrixError()
--> 717 is_diagonalizable, eigenvecs = _is_diagonalizable_with_eigen(M,
    718                        reals_only=reals_only)
    720 if not is_diagonalizable:
    721     raise MatrixError("Matrix is not diagonalizable")

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:449, in
↳ _is_diagonalizable_with_eigen(M, reals_only)
    446 if not M.is_square:
    447     return False, []
--> 449 eigenvecs = M.eigenvects(simplify=True)
    451 for val, mult, basis in eigenvecs:
    452     if reals_only and not val.is_real: # if we have a complex eigenvalu

File /usr/lib/python3/dist-packages/sympy/matrices/matrices.py:381, in
↳ MatrixEigen.eigenvects(self, error_when_incomplete, iszerofunc, **flags)
    380 def eigenvects(self, error_when_incomplete=True, iszerofunc=_iszero,
↳ **flags):
--> 381     return _eigenvects(self, error_when_incomplete=error_when_incomplet,
    382                       iszerofunc=iszerofunc, **flags)

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:431, in
↳ _eigenvects(M, error_when_incomplete, iszerofunc, chop, **flags)
    428 def denom_clean(l):
    429     return [(v / gcd(list(v))).applyfunc(simpfunc) for v in l]
--> 431 ret = [(val, mult, denom_clean(es)) for val, mult, es in ret]
    433 if has_floats:
    434     # if we had floats to start with, turn the eigenvectors to floats
    435     ret = [(val.evalf(chop=chop), mult, [v.evalf(chop=chop) for v in es])
    436            for val, mult, es in ret]
```

```

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:431, in <listcomp>(<
->0)
    428     def denom_clean(l):
    429         return [(v / gcd(list(v))).applyfunc(simpfunc) for v in l]
--> 431     ret = [(val, mult, denom_clean(es)) for val, mult, es in ret]
    433 if has_floats:
    434     # if we had floats to start with, turn the eigenvectors to floats
    435     ret = [(val.evalf(chop=chop), mult, [v.evalf(chop=chop) for v in es])
    436            for val, mult, es in ret]

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:429, in _eigenvects
-><locals>.denom_clean(l)
    428 def denom_clean(l):
--> 429     return [(v / gcd(list(v))).applyfunc(simpfunc) for v in l]

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:429, in <listcomp>(<
->0)
    428 def denom_clean(l):
--> 429     return [(v / gcd(list(v))).applyfunc(simpfunc) for v in l]

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/common.py:2058, in
->MatrixOperations.applyfunc(self, f)
    2055 if not callable(f):
    2056     raise TypeError("`f` must be callable.")
-> 2058 return self._eval_applyfunc(f)

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/common.py:2000, in
->MatrixOperations._eval_applyfunc(self, f)
    1999 def _eval_applyfunc(self, f):
-> 2000     out = self._new(self.rows, self.cols, [f(x) for x in self])
    2001     return out

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/common.py:2000, in
-><listcomp>(.0)
    1999 def _eval_applyfunc(self, f):
-> 2000     out = self._new(self.rows, self.cols, [f(x) for x in self])
    2001     return out

```

```

File /usr/lib/python3/dist-packages/sympy/matrices/utilities.py:72, in
->_simplify(expr)
    70 """ Wrapper to avoid circular imports. """
    71 from sympy.simplify.simplify import simplify
---> 72 return simplify(expr)

```

```

File /usr/lib/python3/dist-packages/sympy/simplify/simplify.py:644, in
->simplify(expr, ratio, measure, rational, inverse, doit, **kwargs)
    642 expr = _bottom_up(expr, lambda w: getattr(w, 'normal', lambda: w)())

```

```

    643 expr = Mul(*powsimp(expr).as_content_primitive())
--> 644 _e = cancel(expr)
    645 expr1 = shorter(_e, _mexpand(_e).cancel()) # issue 6829
    646 expr2 = shorter(together(expr, deep=True), together(expr1, deep=True))

File /usr/lib/python3/dist-packages/sympy/polys/polytools.py:6766, in cancel(f,
↳ _signsimp, *gens, **args)
    6764 if f.has(Piecewise):
    6765     raise PolynomialError()
-> 6766 R, (F, G) = sring((p, q), *gens, **args)
    6767 if not R.ngens:
    6768     if not isinstance(f, (tuple, Tuple)):

File /usr/lib/python3/dist-packages/sympy/polys/rings.py:163, in sring(exprs,
↳ *symbols, **options)
    160 opt = build_options(symbols, options)
    162 # TODO: rewrite this so that it doesn't use expand() (see poly()).
--> 163 reps, opt = _parallel_dict_from_expr(exprs, opt)
    165 if opt.domain is None:
    166     coeffs = sum([ list(rep.values()) for rep in reps ], [])

File /usr/lib/python3/dist-packages/sympy/polys/polyutils.py:326, in
↳ _parallel_dict_from_expr(exprs, opt)
    324 """Transform expressions into a multinomial form. """
    325 if opt.expand is not False:
--> 326     exprs = [ expr.expand() for expr in exprs ]
    328 if any(expr.is_commutative is False for expr in exprs):
    329     raise PolynomialError('non-commutative expressions are not
↳ supported')

File /usr/lib/python3/dist-packages/sympy/polys/polyutils.py:326, in <listcomp> .
↳ 0)
    324 """Transform expressions into a multinomial form. """
    325 if opt.expand is not False:
--> 326     exprs = [ expr.expand() for expr in exprs ]
    328 if any(expr.is_commutative is False for expr in exprs):
    329     raise PolynomialError('non-commutative expressions are not
↳ supported')

File /usr/lib/python3/dist-packages/sympy/core/cache.py:70, in __cacheit.
↳ <locals>.func_wrapper.<locals>.wrapper(*args, **kwargs)
    67 @wraps(func)
    68 def wrapper(*args, **kwargs):
    69     try:
---> 70         retval = cfunc(*args, **kwargs)
    71     except TypeError as e:
    72         if not e.args or not e.args[0].startswith('unhashable type:'):

```

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:3651, in Expr.
    ↪ expand(self, deep, modulus, power_base, power_exp, mul, log, multinomial,
    ↪ basic, **hints)
    3649     if use_hint:
    3650         hint = '_eval_expand_' + hint
-> 3651         expr, hit = Expr._expand_hint(expr, hint, deep=deep, **hints)
    3653 while True:
    3654     was = expr

```

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:3584, in Expr.
    ↪ _expand_hint(expr, hint, deep, **hints)
    3582 sargs = []
    3583 for arg in expr.args:
-> 3584     arg, arghit = Expr._expand_hint(arg, hint, **hints)
    3585     hit |= arghit
    3586     sargs.append(arg)

```

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:3584, in Expr.
    ↪ _expand_hint(expr, hint, deep, **hints)
    3582 sargs = []
    3583 for arg in expr.args:
-> 3584     arg, arghit = Expr._expand_hint(arg, hint, **hints)
    3585     hit |= arghit
    3586     sargs.append(arg)

```

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:3592, in Expr.
    ↪ _expand_hint(expr, hint, deep, **hints)
    3589     expr = expr.func(*sargs)
    3591 if hasattr(expr, hint):
-> 3592     newexpr = getattr(expr, hint)(**hints)
    3593     if newexpr != expr:
    3594         return (newexpr, True)

```

```

File /usr/lib/python3/dist-packages/sympy/core/power.py:1214, in Pow.
    ↪ _eval_expand_multinomial(self, **hints)
    1209     return expand_mul(f*g, deep=False) + n*g*o
    1211 if base.is_number:
    1212     # Efficiently expand expressions of the form (a + b*I)**n
    1213     # where 'a' and 'b' are real numbers and 'n' is integer.
-> 1214     a, b = base.as_real_imag()
    1216     if a.is_Rational and b.is_Rational:
    1217         if not a.is_Integer:

```

```

File /usr/lib/python3/dist-packages/sympy/core/add.py:990, in Add.
    ↪ as_real_imag(self, deep, **hints)
    988 re_part, im_part = [], []
    989 for term in sargs:
--> 990     re, im = term.as_real_imag(deep=deep)

```

```

991     re_part.append(re)
992     im_part.append(im)

```

File /usr/lib/python3/dist-packages/sympy/core/mul.py:849, in Mul.

```

↳as_real_imag(self, deep, **hints)
    847 addterms = S.One
    848 for a in self.args:
--> 849     r, i = a.as_real_imag()
    850     if i.is_zero:
    851         coeffr.append(r)

```

File /usr/lib/python3/dist-packages/sympy/core/power.py:1342, in Pow.

```

↳as_real_imag(self, deep, **hints)
    1337     return S.Zero, (-self.base)**self.exp
    1339 # XXX: This is not totally correct since for x**(p/q) with
    1340 #       x being imaginary there are actually q roots, but
    1341 #       only a single one is returned from here.
-> 1342 r = self.func(self.func(re_e, 2) + self.func(im_e, 2), S.Half)
    1344 t = atan2(im_e, re_e)
    1346 rp, tp = self.func(r, self.exp), t*self.exp

```

File /usr/lib/python3/dist-packages/sympy/core/cache.py:70, in __cacheit.

```

↳<locals>.func_wrapper.<locals>.wrapper(*args, **kwargs)
    67 @wraps(func)
    68 def wrapper(*args, **kwargs):
    69     try:
---> 70         retval = cfunc(*args, **kwargs)
    71     except TypeError as e:
    72         if not e.args or not e.args[0].startswith('unhashable type:'):

```

File /usr/lib/python3/dist-packages/sympy/core/power.py:371, in Pow.__new__(cls, b, e, evaluate)

```

↳b, e, evaluate)
    367     if s.is_Number and s and den == \
    368         log(-factor_terms(b, sign=False)) + s*S.ImaginaryUnit*S
↳Pi:
    369         return S.Exp1**(c*num)
--> 371 obj = b._eval_power(e)
    372 if obj is not None:
    373     return obj

```

File /usr/lib/python3/dist-packages/sympy/core/add.py:458, in Add.

```

↳_eval_power(self, e)
    456 from .evalf import pure_complex
    457 from .relational import is_eq
--> 458 if len(self.args) == 2 and any(_.is_infinite for _ in self.args):
    459     if e.is_zero is False and is_eq(e, S.One) is False:
    460         # looking for literal a + I*b
    461         a, b = self.args

```

```

File /usr/lib/python3/dist-packages/sympy/core/add.py:458, in <genexpr>(.0)
    456 from .evalf import pure_complex
    457 from .relational import is_eq
--> 458 if len(self.args) == 2 and any(_is_infinite for _ in self.args):
    459     if e.is_zero is False and is_eq(e, S.One) is False:
    460         # looking for literal a + I*b
    461         a, b = self.args

```

```

File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:483, in
↳make_property.<locals>.getit(self)
    481 if self._assumptions is self.default_assumptions:
    482     self._assumptions = self.default_assumptions.copy()
--> 483 return _ask(fact, self)

```

```

File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:539, in _ask(fact
↳obj)
    537 handler_i = handler_map.get(fact_i)
    538 if handler_i is not None:
--> 539     fact_i_value = handler_i(obj)
    541 # If we get a new value for fact_i then we should update our knowledge
    542 # of fact_i as well as any related facts that can be inferred using the
    543 # inference rules connecting the fact_i and any other fact values that
    544 # are already known.
    545 if fact_i_value is not None:

```

```

File /usr/lib/python3/dist-packages/sympy/core/power.py:825, in Pow.
↳_eval_is_composite(self)
    821 def _eval_is_composite(self):
    822     """
    823     A power is composite if both base and exponent are greater than 1
    824     """
--> 825     if (self.base.is_integer and self.exp.is_integer and
    826         ((self.base - 1).is_positive and (self.exp - 1).is_positive or
    827         (self.base + 1).is_negative and self.exp.is_positive and self.
↳exp.is_even)):
    828         return True

```

```

File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:483, in
↳make_property.<locals>.getit(self)
    481 if self._assumptions is self.default_assumptions:
    482     self._assumptions = self.default_assumptions.copy()
--> 483 return _ask(fact, self)

```

```

File /usr/lib/python3/dist-packages/sympy/core/assumptions.py:539, in _ask(fact
↳obj)
    537 handler_i = handler_map.get(fact_i)
    538 if handler_i is not None:

```



```

--> 539     fact_i_value = handler_i(obj)
      541 # If we get a new value for fact_i then we should update our knowledge
      542 # of fact_i as well as any related facts that can be inferred using the
      543 # inference rules connecting the fact_i and any other fact values that
      544 # are already known.
      545 if fact_i_value is not None:

```

File /usr/lib/python3/dist-packages/sympy/core/add.py:752, in Add.

```

↪ _eval_is_extended_positive(self)
      750 def _eval_is_extended_positive(self):
      751     if self.is_number:
--> 752         return super()._eval_is_extended_positive()
      753     c, a = self.as_coeff_Add()
      754     if not c.is_zero:

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:891, in Expr.

```

↪ _eval_is_extended_positive(self)
      890 def _eval_is_extended_positive(self):
--> 891     return self._eval_is_extended_positive_negative(positive=True)

```

File /usr/lib/python3/dist-packages/sympy/core/expr.py:870, in Expr.

```

↪ _eval_is_extended_positive_negative(self, positive)
      867 if n2 is S.NaN:
      868     return None
--> 870 f = self.evalf(2)
      871 if f.is_Float:
      872     match = f, S.Zero

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1659, in EvalfMixin.

```

↪ evalf(self, n, subs, maxn, chop, strict, quad, verbose)
      1657     options['quad'] = quad
      1658 try:
-> 1659     result = evalf(self, prec + 4, options)
      1660 except NotImplementedError:
      1661     # Fall back to the ordinary evalf
      1662     if hasattr(self, 'subs') and subs is not None: # issue 20291

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,

```

↪ options)
      1494 try:
      1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
      1497 except KeyError:
      1498     # Fall back to ordinary evalf if possible
      1499     if 'subs' in options:

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in evalf_add(v, u,

```

↪ prec, options)

```

```

599 while 1:
600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
603     n = terms.count(S.ComplexInfinity)
604     if n >= 2:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in <listcomp>(.)
599 while 1:
600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
603     n = terms.count(S.ComplexInfinity)
604     if n >= 2:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳ options)
1494 try:
1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
1497 except KeyError:
1498     # Fall back to ordinary evalf if possible
1499     if 'subs' in options:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:698, in evalf_mul(v,
↳ prec, options)
696 elif i == last and arg is S.One:
697     continue
--> 698 re, im, re_acc, im_acc = evalf(arg, working_prec, options)
699 if re and im:
700     complex_factors.append((re, im, re_acc, im_acc))

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳ options)
1494 try:
1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
1497 except KeyError:
1498     # Fall back to ordinary evalf if possible
1499     if 'subs' in options:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:313, in evalf_im(expr,
↳ prec, options)
312 def evalf_im(expr: 'im', prec: int, options: OPT_DICT) -> TMP_RES:
--> 313     return get_complex_part(expr.args[0], 1, prec, options)

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:293, in
↳ get_complex_part(expr, no, prec, options)
291 i = 0
292 while 1:

```

```

--> 293     res = evalf(expr, workprec, options)
      294     if res is S.ComplexInfinity:
      295         return fnan, None, prec, None

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↪options)

```

      1494 try:
      1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
      1497 except KeyError:
      1498     # Fall back to ordinary evalf if possible
      1499     if 'subs' in options:

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:650, in evalf_mul(v,
↪prec, options)

```

      648 from .numbers import Float
      649 for arg in args:
--> 650     result = evalf(arg, prec, options)
      651     if result is S.ComplexInfinity:
      652         special.append(result)

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↪options)

```

      1494 try:
      1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
      1497 except KeyError:
      1498     # Fall back to ordinary evalf if possible
      1499     if 'subs' in options:

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:857, in evalf_pow(v,
↪prec, options)

```

      854     return finalize_complex(re, im, target_prec)
      855     return mpf_exp(yre, target_prec), None, target_prec, None
--> 857 xre, xim, _, _ = evalf(base, prec + 5, options)
      858 # 0**y
      859 if not (xre or xim):

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↪options)

```

      1494 try:
      1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
      1497 except KeyError:
      1498     # Fall back to ordinary evalf if possible
      1499     if 'subs' in options:

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:698, in evalf_mul(v,
↳prec, options)
    696 elif i == last and arg is S.One:
    697     continue
--> 698 re, im, re_acc, im_acc = evalf(arg, working_prec, options)
    699 if re and im:
    700     complex_factors.append((re, im, re_acc, im_acc))

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)
    1494 try:
    1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
    1497 except KeyError:
    1498     # Fall back to ordinary evalf if possible
    1499     if 'subs' in options:

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in evalf_add(v,
↳prec, options)
    599 while 1:
    600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
    603     n = terms.count(S.ComplexInfinity)
    604     if n >= 2:

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in <listcomp>(.)
    599 while 1:
    600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
    603     n = terms.count(S.ComplexInfinity)
    604     if n >= 2:

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)
    1494 try:
    1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
    1497 except KeyError:
    1498     # Fall back to ordinary evalf if possible
    1499     if 'subs' in options:

```

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:698, in evalf_mul(v,
↳prec, options)
    696 elif i == last and arg is S.One:
    697     continue
--> 698 re, im, re_acc, im_acc = evalf(arg, working_prec, options)
    699 if re and im:
    700     complex_factors.append((re, im, re_acc, im_acc))

```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)

```
1494 try:
1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
1497 except KeyError:
1498     # Fall back to ordinary evalf if possible
1499     if 'subs' in options:
```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in evalf_add(v,
↳prec, options)

```
599 while 1:
600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
603     n = terms.count(S.ComplexInfinity)
604     if n >= 2:
```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in <listcomp>(.)

```
599 while 1:
600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
603     n = terms.count(S.ComplexInfinity)
604     if n >= 2:
```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)

```
1494 try:
1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
1497 except KeyError:
1498     # Fall back to ordinary evalf if possible
1499     if 'subs' in options:
```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:650, in evalf_mul(v,
↳prec, options)

```
648 from .numbers import Float
649 for arg in args:
--> 650     result = evalf(arg, prec, options)
651     if result is S.ComplexInfinity:
652         special.append(result)
```

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)

```
1494 try:
1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
1497 except KeyError:
```

```

1498     # Fall back to ordinary evalf if possible
1499     if 'subs' in options:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:857, in evalf_pow(v,
↳prec, options)
    854         return finalize_complex(re, im, target_prec)
    855     return mpf_exp(yre, target_prec), None, target_prec, None
--> 857 xre, xim, _, _ = evalf(base, prec + 5, options)
    858 # 0**y
    859 if not (xre or xim):

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)
    1494 try:
    1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
    1497 except KeyError:
    1498     # Fall back to ordinary evalf if possible
    1499     if 'subs' in options:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in evalf_add(v,
↳prec, options)
    599 while 1:
    600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
    603     n = terms.count(S.ComplexInfinity)
    604     if n >= 2:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:602, in <listcomp>(.0)
    599 while 1:
    600     options['maxprec'] = min(oldmaxprec, 2*prec)
--> 602     terms = [evalf(arg, prec + 10, options) for arg in v.args]
    603     n = terms.count(S.ComplexInfinity)
    604     if n >= 2:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1496, in evalf(x, prec,
↳options)
    1494 try:
    1495     rf = evalf_table[type(x)]
-> 1496     r = rf(x, prec, options)
    1497 except KeyError:
    1498     # Fall back to ordinary evalf if possible
    1499     if 'subs' in options:

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:650, in evalf_mul(v,
↳prec, options)
    648 from .numbers import Float
    649 for arg in args:

```

```

--> 650     result = evalf(arg, prec, options)
      651     if result is S.ComplexInfinity:
      652         special.append(result)

File /usr/lib/python3/dist-packages/sympy/core/evalf.py:1545, in evalf(x, prec,
↪options)
      1543         chop_prec -= 1
      1544     r = chop_parts(r, chop_prec)
-> 1545 if options.get("strict"):
      1546     check_target(x, r, prec)
      1547 return r

KeyboardInterrupt:

```

```

[ ]: U.diagonalize() # l'exécution reste rapide si le calcul des éléments propres
↪n'exige pas de se placer dans des corps d'extension compliqués.

```

```

[ ]: (Matrix([
[-1, -1, -1, 1],
[ 1,  0,  0, 1],
[ 0,  1,  0, 1],
[ 0,  0,  1, 1]]),
Matrix([
[0, 0, 0, 0],
[0, 0, 0, 0],
[0, 0, 0, 0],
[0, 0, 0, 4]]))

```

Pour une illustration moins encombrée des fonctionnalités de sympy, nous allons générer des matrices dont les éléments propres sont décidés dès le début. On construit d'abord une matrice inversible de déterminant égal à 1 pour l'utiliser comme matrice de passage et on conjugue avec deux matrices, l'une diagonale et l'autre triangulaire supérieure (une réduite de Jordan non diagonalisable)

```

[143]: P=randMatrix(4,4,-1,2)
while P.det() != 1 :
    P=randMatrix(4,4,-1,2)
P

```

```

[143]: 
$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 1 & 2 & -1 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & -1 & 2 & 0 \end{bmatrix}$$


```

```

[144]: T=diag(1,Matrix(3,3,[2,1,0,0,2,1,0,0,2])) ; T

```

```

[144]: 
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$


```

```
[145]: A=P*diag(1,1,0,-1)*P**(-1) ; B=P*T*P**(-1) ; display(A,B)
```

$$\begin{bmatrix} 1 & -2 & 2 & -2 \\ -2 & 10 & -11 & 7 \\ -2 & 8 & -9 & 6 \\ 0 & -2 & 2 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 10 & -12 & 8 \\ 0 & 2 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ -1 & 5 & -7 & 5 \end{bmatrix}$$

```
[146]: factor(A.charpoly(x))
```

```
[146]: PurePoly( $x^4 - x^3 - x^2 + x$ ,  $x$ , domain =  $\mathbb{Z}$ )
```

```
[147]: factor(A.charpoly(x).as_expr())
```

```
[147]:  $x(x-1)^2(x+1)$ 
```

```
[148]: factor(B.charpoly(x).as_expr())
```

```
[148]:  $(x-2)^3(x-1)$ 
```

```
[149]: B.eigenvals() # retourne un dictionnaire Python, chaque valeur propre est ↵
↵associé à sa multiplicité
```

```
[149]: {1: 1, 2: 3}
```

```
[150]: A.eigenvects() # retourne une liste de tuple, chaque tuple contient une vap, sa ↵
↵mult et une base du sep
```

```
[150]: [(-1,
1,
[Matrix([
[0],
[1],
[1],
[0]])]),
(0,
1,
[Matrix([
[1],
[-1/2],
[0],
[1]])]),
(1,
2,
[Matrix([
[-1],
```



```
[ 1],
[ 1],
[ 0]])],
Matrix([
[-1],
[-1],
[ 0],
[ 1]])))]
```

```
[ ]: A.is_diagonalizable() # retourne une valeur booléenne, vrai si A est
↳diagonalisable, peut être très lent au même titre que 'diagonalize'
```

```
[ ]: True
```

```
[153]: B.eigenvects() # permet juger de la diagonalisabilité en se basant sur le cours
```

```
[153]: [(1,
1,
[Matrix([
[-1],
[-1],
[ 0],
[ 1]])]),
(2,
3,
[Matrix([
[ 0],
[-2],
[-1],
[ 1]])]))]
```

```
[154]: A.diagonalize() # tuple formé de deux matrices : une matrice de passage et
↳matrice diagonale.
```

```
[154]: (Matrix([
[0, 2, -1, -1],
[1, -1, 1, -1],
[1, 0, 1, 0],
[0, 2, 0, 1]]),
Matrix([
[-1, 0, 0, 0],
[ 0, 0, 0, 0],
[ 0, 0, 1, 0],
[ 0, 0, 0, 1]]))
```

```
[155]: P,D=A.diagonalize() ; display(P,D) # récupérer P et D
```

$$\begin{bmatrix} 0 & 2 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
[156]: B.is_diagonalizable() # bien sûr que non
```

```
[156]: False
```

```
[157]: B.jordan_form() # permet de trigonaliser, retourne une matrice de passage et
      ↪ une matrice triangulaire, peut être très lent
```

```
[157]: (Matrix([
      [-1, 0, 4, 2],
      [-1, 4, 0, 1],
      [ 0, 2, 1, 0],
      [ 1, -2, 3, 0]]),
      Matrix([
      [1, 0, 0, 0],
      [0, 2, 1, 0],
      [0, 0, 2, 1],
      [0, 0, 0, 2]]))
```

```
[ ]: P,T=B.jordan_form() ; display(P,T) # récupérer les matrices P et T
```

$$\begin{bmatrix} -\frac{1}{2} & \frac{1}{2} & -2 & \frac{3}{2} \\ -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & 1 \\ 0 & -\frac{1}{2} & 1 & 0 \\ 1 & 0 & -\frac{1}{2} & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

```
[ ]: P*T*P**(-1) == B # verification de l'égalité de deux matrices, comparaison
      ↪ toute bête grâce à sympy
```

```
[ ]: True
```

3 Exemples pratiques

3.0.1 Exemple 1 : réduction d'une matrice compagne

Sympy fournit la méthode `companion` qui permet de construire la matrice compagne d'un polynôme unitaire donné.

```
[379]: x=Symbol('x')
P=Poly((x-1)**3*x*(x+1),x)
C=Matrix.companion(P)
display(C)
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix}$$

N.B : Il n'est pas difficile de former la matrice compagne d'un polynôme sans utiliser la méthode `companion` comme le montre le code suivant :

```
[380]: Matrix.hstack(banded({-1:(1,1,1,1)}),-Matrix(5,1,[P.nth(k) for k in range(5)]))
```

```
[380]:
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix}$$

```
[16]: factor(C.charpoly(x).as_expr())
```

```
[16]:
```

$$x(x-1)^3(x+1)$$

```
[17]: C.diagonalize() # oups, une matrice compagne n'est diagonalisable que si ...
```

```
-----
MatrixError                                Traceback (most recent call last)
Cell In [17], line 1
----> 1 C.diagonalize() # oups, une matrice compagne n'est diagonalisable que s
↪...

File /usr/lib/python3/dist-packages/sympy/matrices/matrices.py:388, in
↪MatrixEigen.diagonalize(self, reals_only, sort, normalize)
    387 def diagonalize(self, reals_only=False, sort=False, normalize=False):
--> 388     return _diagonalize(self, reals_only=reals_only, sort=sort,
    389                        normalize=normalize)

File /usr/lib/python3/dist-packages/sympy/matrices/eigen.py:721, in
↪_diagonalize(M, reals_only, sort, normalize)
    717 is_diagonalizable, eigenvecs = _is_diagonalizable_with_eigen(M,
```

```

718             reals_only=reals_only)
720 if not is_diagonalizable:
--> 721     raise MatrixError("Matrix is not diagonalizable")
723 if sort:
724     eigenvecs = sorted(eigenvecs, key=default_sort_key)

MatrixError: Matrix is not diagonalizable

```

```
[18]: Q,T=C.jordan_form() ; display(Q,T)
```

$$\begin{bmatrix} 0 & -1 & 0 & 0 & 0 \\ -1 & 2 & 1 & -1 & 1 \\ 3 & 0 & -1 & 0 & 1 \\ -3 & -2 & -1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Sachant a, b et c étant des paramètres *distincts* quelconques, on veut réduire la matrice compagne du polynôme $P = (x - a)(X - b)^2(x - c)^3$ ainsi que sa transposée tout en simplifiant les matrices obtenues pour bien visualiser leur forme générale.

```
[616]: a,b,c,x,y=symbols('a b c x y')
P=poly((x-a)*(x-b)**2*(x-c)**3,x)
C=Matrix.companion(P)
display(P)
display(C)
```

Poly($x^6 + (-a - 2b - 3c)x^5 + (2ab + 3ac + b^2 + 6bc + 3c^2)x^4 + (-ab^2 - 6abc - 3ac^2 - 3b^2c - 6bc^2 - c^3)x^3 + (3a^2b + 3ab^2 + 3ac^2 + 3b^2c + 3c^3)x^2 + (-a^3 - 3a^2b - 3ab^2 - 3ac^2 - 3b^2c - 3c^3)x + a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + 3a^3c + 6a^2bc + 3ab^2c + 3ac^3 + 6abc^2 + 3b^2c^2 + 3c^3$)

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & -ab^2c^3 \\ 1 & 0 & 0 & 0 & 0 & 3ab^2c^2 + 2abc^3 + b^2c^3 \\ 0 & 1 & 0 & 0 & 0 & -3ab^2c - 6abc^2 - ac^3 - 3b^2c^2 - 2bc^3 \\ 0 & 0 & 1 & 0 & 0 & ab^2 + 6abc + 3ac^2 + 3b^2c + 6bc^2 + c^3 \\ 0 & 0 & 0 & 1 & 0 & -2ab - 3ac - b^2 - 6bc - 3c^2 \\ 0 & 0 & 0 & 0 & 1 & a + 2b + 3c \end{bmatrix}$$

```
[613]: Q,T=C.jordan_form(C)
display(Q,T)
```

$$\begin{bmatrix} -b^2c^3 & -abc^3 & ac^3 & -ab^2c^2 & ab^2c^2 \\ 3b^2c^2 + 2bc^3 & ac^3 - b(-3ac^2 - c^3) & -3ac^2 - c^3 & 2ab^2c + c^2 \cdot (2ab + b^2) & -ab^2 - c(2ab + b^2) \\ -3b^2c - 6bc^2 - c^3 & -3ac^2 - b(3ac + 3c^2) - c^3 & 3ac + 3c^2 & -ab^2 + c^2(-a - 2b) - 2c(2ab + b^2) & 2ab + b^2 - c^2 \\ b^2 + 6bc + 3c^2 & 3ac - b(-a - 3c) + 3c^2 & -a - 3c & 2ab + b^2 + c^2 - 2c(-a - 2b) & -a - 2b - 3c \\ -2b - 3c & -a - b - 3c & 1 & -a - 2b - 2c & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 0 & 0 & 0 & 0 & 0 \\ 0 & b & 1 & 0 & 0 & 0 \\ 0 & 0 & b & 0 & 0 & 0 \\ 0 & 0 & 0 & c & 1 & 0 \\ 0 & 0 & 0 & 0 & c & 1 \\ 0 & 0 & 0 & 0 & 0 & c \end{bmatrix}$$

[614]: `simplify(C*Q-Q*T) # vérification, 'C*Q == Q*T' conviendrait aussi`

[614]:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Même chose pour la matrice C^T maintenant

[630]: `QT,TT=C.T.jordan_form()
QT=QT.applyfunc(simplify)
display(QT,TT)`

$$\begin{bmatrix} \frac{1}{a^5} & -\frac{1}{b^3} & \frac{5}{b^4} & \frac{1}{c} & -\frac{4}{c^2} & \frac{10}{c^3} \\ \frac{1}{a^4} & -\frac{1}{b^2} & \frac{4}{b^3} & 1 & -\frac{3}{c} & \frac{6}{c^2} \\ \frac{1}{a^3} & -\frac{1}{b} & \frac{3}{b^2} & c & -2 & \frac{3}{c} \\ \frac{1}{a^2} & -1 & \frac{2}{b} & c^2 & -c & 1 \\ \frac{1}{a} & -b & 1 & c^3 & 0 & 0 \\ 1 & -b^2 & 0 & c^4 & c^3 & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 0 & 0 & 0 & 0 & 0 \\ 0 & b & 1 & 0 & 0 & 0 \\ 0 & 0 & b & 0 & 0 & 0 \\ 0 & 0 & 0 & c & 1 & 0 \\ 0 & 0 & 0 & 0 & c & 1 \\ 0 & 0 & 0 & 0 & 0 & c \end{bmatrix}$$

La réponse n'est pas satisfaisante à cause des fractions dans 'QT' qui ne tiennent pas compte du fait que l'un des coefficients a, b ou c peut être nul. Les vecteurs colonnes V_1, V_2, V_3, V_4, V_5 de QT sont liées par les relations

$$(C - aI)V_1 = 0 \quad (1)$$

$$(C - bI)V_2 = 0 \quad (2)$$

$$(C - bI)V_3 = V_2 \quad (3)$$

$$(C - cI)V_4 = 0 \quad (4)$$

$$(C - cI)V_5 = V_4 \quad (5)$$

$$(C - cI)V_6 = V_5 \quad (6)$$

Ces relations restent inchangées si on multiplie V_2 et V_3 par un même scalaire et ensuite si on ajoute à V_3 un vecteur colinéaire à V_2 . Une observation similaire peut être faite pour les vecteurs V_4, V_5 et V_6 . Ce qui nous permet de remplacer 'QT' par une matrice plus «générique»

```
[631]: Q_temp=QT[:,:] # on crée une copie de QT pour ne pas avoir à exécuter
      ↪ l'instruction précédente à chaque modification
Q_temp[:,0] *= a**5
Q_temp[:,1] *= -b**3
Q_temp[:,2] = -b**3*Q_temp[:,2]+5/b*Q_temp[:,1]
Q_temp[:,3] *= c
Q_temp[:,4] = c*Q_temp[:,4]+4/c*Q_temp[:,3]
Q_temp[:,5] = c*Q_temp[:,5]-10/c**2*Q_temp[:,3]+4/c*Q_temp[:,4]
display(Q_temp)
```

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ a & b & 1 & c & 1 & 0 \\ a^2 & b^2 & 2b & c^2 & 2c & 1 \\ a^3 & b^3 & 3b^2 & c^3 & 3c^2 & 3c \\ a^4 & b^4 & 4b^3 & c^4 & 4c^3 & 6c^2 \\ a^5 & b^5 & 5b^4 & c^5 & 5c^4 & 10c^3 \end{bmatrix}$$

On observe que les colonnes relatives à une même valeur propre λ de C^T peuvent être déduites simplement par dérivations successives par rapport à λ de la colonne

$$\begin{pmatrix} 1 \\ \lambda \\ \lambda^2 \\ \lambda^3 \\ \lambda^4 \\ \lambda^5 \end{pmatrix}$$

```
[633]: QT=Q_temp
simplify(C.T*QT - QT*TT)
```

```
[633]: 
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

```

Exercice : écrire une fonction Python qui prend en argument un Poly et retourne les matrices C_P^T , T et Q telles que $C_P^T = QTQ^{-1}$, Q étant construite selon les observations précédentes. On introduira les fonctions suivantes

```
[649]: def Vand(n,x) :
      return Matrix(n,1,[x**i for i in range(n)])
def MatDiffVand(n,k,x) :
    y_gen=symbols('ygen')
    U=Vand(n,y_gen)
    V_temp = U
    if k == 1 :
```

```

    return V_temp.subs(y_gen,x)
else :
    for i in range(2,k+1) :
        U=S(1)/(i-1)*U.diff(y_gen)
        V_temp=V_temp.hstack(V_temp, U)
    return V_temp.subs(y_gen,x)
display(Vand(9,x))
display(MatDiffVand(9,5,x))
display(MatDiffVand(9,5,1+I).applyfunc(expand))

```

$$\begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \\ x^4 \\ x^5 \\ x^6 \\ x^7 \\ x^8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ x & 1 & 0 & 0 & 0 \\ x^2 & 2x & 1 & 0 & 0 \\ x^3 & 3x^2 & 3x & 1 & 0 \\ x^4 & 4x^3 & 6x^2 & 4x & 1 \\ x^5 & 5x^4 & 10x^3 & 10x^2 & 5x \\ x^6 & 6x^5 & 15x^4 & 20x^3 & 15x^2 \\ x^7 & 7x^6 & 21x^5 & 35x^4 & 35x^3 \\ x^8 & 8x^7 & 28x^6 & 56x^5 & 70x^4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1+i & 1 & 0 & 0 & 0 \\ 2i & 2+2i & 1 & 0 & 0 \\ -2+2i & 6i & 3+3i & 1 & 0 \\ -4 & -8+8i & 12i & 4+4i & 1 \\ -4-4i & -20 & -20+20i & 20i & 5+5i \\ -8i & -24-24i & -60 & -40+40i & 30i \\ 8-8i & -56i & -84-84i & -140 & -70+70i \\ 16 & 64-64i & -224i & -224-224i & -280 \end{bmatrix}$$

3.0.2 Exemple 2 : une méthode de calcul du polynôme minimal

On considère une matrice carrée $A \in \mathcal{M}_d(\mathbb{K})$. On rappelle que si $r = \deg \pi_A$ alors $(I_d, A, \dots, A^{r-1})$ est une base de l'algèbre $\mathbb{K}[A]$ et qu'en particulier c'est une famille libre maximale extraite de la famille $(A^k)_{k \in \mathbb{N}}$. Soit M la matrice de taille $(d^2, d+1)$ dont les colonnes sont formées des coordonnées des matrices $I, A, \dots, A^d$ dans la base canonique de $\mathcal{M}_d(\mathbb{K})$. Justifier l'existence d'une

matrice inversible $Q \in \mathcal{M}_{d^2}(\mathbb{K})$ telle que QM soit de la forme

$$QM = \begin{pmatrix} 1 & \cdots & 0 & a_0 & * & \cdots & * \\ \vdots & \ddots & \vdots & \vdots & \vdots & & \vdots \\ 0 & \cdots & 1 & a_{r-1} & * & \cdots & * \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & & & \vdots & \vdots & & \vdots \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \end{pmatrix}$$

On peut justifier qu'alors $P = X^d - a_{d-1}X^{d-1} - \cdots - a_1X - a_0$ est le polynôme minimal de A .

Sous SymPy, il suffit de former la matrice M et d'utiliser la méthode `rref` pour obtenir une matrice de la forme $M_J = QM$ ci-dessus.

```
[40]: x=symbols('x')
A=randMatrix(5,5,-2,2) ; display(A)
M=Matrix([[*A**k] for k in range(6)]).T
display(M)
display(M.rref()[0][0:5,:])
print("Polynôme minimal calculé :")
display(Poly(x**5-sum(M.rref()[0][i,5]*x**i for i in range(5))))
print("Polynôme caractéristique :")
display(A.charpoly(x).as_poly())
```

$$\begin{bmatrix} -1 & 1 & 1 & 2 & -2 \\ 2 & 2 & 1 & -1 & 2 \\ 2 & 1 & 1 & 1 & 0 \\ 1 & 2 & 1 & 2 & -2 \\ 2 & 2 & -1 & 2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -1 & 3 & 9 & 32 & 126 \\ 0 & 1 & 2 & 8 & 47 & 194 \\ 0 & 1 & 5 & 8 & 28 & 151 \\ 0 & 2 & -2 & 5 & 32 & 85 \\ 0 & -2 & 0 & 2 & -12 & -34 \\ 0 & 2 & 7 & 24 & 109 & 473 \\ 1 & 2 & 9 & 41 & 157 & 636 \\ 0 & 1 & 2 & 21 & 113 & 445 \\ 0 & -1 & 5 & 21 & 58 & 274 \\ 0 & 2 & 2 & -6 & -8 & -20 \\ 0 & 2 & 3 & 19 & 83 & 330 \\ 0 & 1 & 7 & 26 & 104 & 459 \\ 1 & 1 & 5 & 25 & 82 & 316 \\ 0 & 1 & 6 & 8 & 45 & 230 \\ 0 & 0 & -4 & -4 & -2 & -48 \\ 0 & 1 & 3 & 22 & 74 & 299 \\ 0 & 2 & 6 & 21 & 104 & 445 \\ 0 & 1 & 8 & 20 & 65 & 320 \\ 1 & 2 & 1 & 6 & 63 & 207 \\ 0 & -2 & -2 & 4 & -14 & -66 \\ 0 & 2 & 2 & 23 & 91 & 347 \\ 0 & 2 & 9 & 27 & 114 & 512 \\ 0 & -1 & 5 & 25 & 73 & 330 \\ 0 & 2 & 5 & 2 & 56 & 261 \\ 1 & 0 & -4 & 4 & 4 & -66 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -34 \\ 0 & 1 & 0 & 0 & 0 & -8 \\ 0 & 0 & 1 & 0 & 0 & 11 \\ 0 & 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Polynôme minimal calculé :

$$\text{Poly}(x^5 - 4x^4 + x^3 - 11x^2 + 8x + 34, x, \text{domain} = \mathbb{Z})$$

Polynôme caractéristique :

$$\text{Poly}(x^5 - 4x^4 + x^3 - 11x^2 + 8x + 34, x, \text{domain} = \mathbb{Z})$$

Exercice : Écrire une fonction Python qui automatise toute la procédure. _____

Exemple 3: une fonction plus rapide de diagonalisation Écrire, en se basant sur la méthode sympy `eigenvects`, une fonction qui prend en argument une matrice M capable de détecter si M est diagonalisable et qui retourne dans ce cas une matrice de passage P et une matrice diagonale D telle $M = PDP^{-1}$.

```
[19]: A=randMatrix(3,3,-5,5) ; display(A)
```

$$\begin{bmatrix} 3 & 5 & -4 \\ -4 & -2 & 4 \\ -3 & 4 & -4 \end{bmatrix}$$

[20]: A.eigenvects()

```
[20]: [(-1 - 21/((-1/2 - sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) - (-1/2 -
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3,
1,
[Matrix([
[ 59/169 + (-1/2 - sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/13 - 4*(-1 -
21/((-1/2 - sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) - (-1/2 -
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3)**2/169 + 63/(13*(-1/2 -
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3))],
[171/169 - 3*(-1 - 21/((-1/2 - sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) -
(-1/2 - sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3)**2/169 - 21/(13*(-1/2 -
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) - (-1/2 -
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/39],
[
1]])]),
(-1 - (-1/2 + sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3 - 21/((-1/2 +
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)),
1,
[Matrix([
[ 59/169 + 63/(13*(-1/2 + sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) - 4*(-1
- (-1/2 + sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3 - 21/((-1/2 +
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)))**2/169 + (-1/2 +
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/13],
[171/169 - (-1/2 + sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/39 -
21/(13*(-1/2 + sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)) - 3*(-1 - (-1/2 +
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)/3 - 21/((-1/2 +
sqrt(3)*I/2)*(27*sqrt(1961) + 1296)**(1/3)))**2/169],
[
1]])]),
(-(27*sqrt(1961) + 1296)**(1/3)/3 - 21/(27*sqrt(1961) + 1296)**(1/3) - 1,
1,
[Matrix([
[ -4*(-(27*sqrt(1961) + 1296)**(1/3)/3 - 21/(27*sqrt(1961) + 1296)**(1/3) -
1)**2/169 + 59/169 + 63/(13*(27*sqrt(1961) + 1296)**(1/3)) + (27*sqrt(1961) +
1296)**(1/3)/13],
[-3*(-(27*sqrt(1961) + 1296)**(1/3)/3 - 21/(27*sqrt(1961) + 1296)**(1/3) -
1)**2/169 - (27*sqrt(1961) + 1296)**(1/3)/39 - 21/(13*(27*sqrt(1961) +
1296)**(1/3)) + 171/169],
[
1]])])]
```

```
[24]: def simple_diag (M) :
      """
      adaptée à partir de la méthode 'diagonalize' elle même.
      Vive le logiciel libre !
      """

      if not M.is_square:
          raise NonSquareMatrixError()

      p_cols, diag = [], []
      for val, mult, basis in M.eigenvecs() :
          diag += [val] * mult
          p_cols += basis

      P_cols=Matrix.hstack(*p_cols) # P_cols a pour colonnes tous les vecteurs
      ↪trouvés par 'eigenvecs'

      if P_cols.cols < M.cols : # si P_cols a moins de colonnes que A c'est qu'il
      ↪n'y a pas de base formée de vep.
          raise MatrixError("Matrice non diagonalisable")
      else :
          return P_cols, M.diag(*diag)
```

```
[46]: P,D=simple_diag(A) ; display(P,D)
```

$$\begin{bmatrix} \frac{59}{169} + \frac{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{13} - \frac{4\left(-1 - \frac{21}{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} - \frac{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{3}\right)^2}{169} + \frac{63}{13\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} & \frac{59}{169} \\ \frac{171}{169} - \frac{3\left(-1 - \frac{21}{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} - \frac{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{3}\right)^2}{169} - \frac{21}{13\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} - \frac{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{39} & \frac{171}{169} \\ 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -1 - \frac{21}{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} - \frac{\left(-\frac{1}{2} - \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{3} & 0 \\ 0 & -1 - \frac{\left(-\frac{1}{2} + \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}}{3} - \frac{21}{\left(-\frac{1}{2} + \frac{\sqrt{3}i}{2}\right)^3 \sqrt[3]{27\sqrt{1961+1296}}} \\ 0 & 0 \end{bmatrix}$$

Exercice : Réécrire la fonction `simple_diag` de telle sorte qu'elle donne des noms symboliques aux valeurs propres et exprime la matrice de passage P en fonctions de ces noms.

3.0.3 Exemple 4 : réduction d'une matrice à paramètres

Réduire la matrice suivante

$$A = \begin{pmatrix} a-2c & -a+b+3c & 4c & a-b-3c \\ 0 & b & 0 & 0 \\ -c & c & a+2c & -c \\ 0 & c & 0 & b \end{pmatrix}$$

a, b, c étant des paramètres réels.

```
[79]: a,b,c=symbols('a b c')
A=Matrix([
[a - 2*c, -a + b + 3*c, 4*c, a - b - 3*c],
[ 0, b, 0, 0],
[-c, c, a + 2*c, -c],
[ 0, c, 0, b]])
display(A)
```

$$\begin{bmatrix} a-2c & -a+b+3c & 4c & a-b-3c \\ 0 & b & 0 & 0 \\ -c & c & a+2c & -c \\ 0 & c & 0 & b \end{bmatrix}$$

On teste avec `jordan_form`

```
[80]: Q,T=A.jordan_form()
```

```
[81]: display(Q,T)
```

$$\begin{bmatrix} -2c & 1 & 4c - \frac{(-a+b+3c)(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} & -\frac{(a-b-2c)(a^2-2ab-ac+b^2+bc-4c^2)}{2c^2} & -\frac{a^2-2ab-ac+b^2+bc-4c^2}{2c^2} \\ 0 & 0 & 0 & -\frac{a^3-3a^2b+3ab^2-b^3}{2ac^2-2bc^2-2c^3} \\ -c & 0 & a-b+2c - \frac{c(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} + \frac{a^2-2ab-ac+b^2+bc-4c^2}{2c} & 1 \\ 0 & 0 & -\frac{c(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 1 \\ 0 & 0 & 0 & b \end{bmatrix}$$

Apparemment la matrice A se réduit sous la forme

$$T = \begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 1 \\ 0 & 0 & 0 & b \end{bmatrix}$$

mais en observant la matrice de passage

$$Q = \begin{bmatrix} -2c & 1 & 4c - \frac{(-a+b+3c)(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} & -\frac{(a-b-2c)(a^2-2ab-ac+b^2+bc-4c^2)}{2c^2} & -\frac{a^2-2ab-ac+b^2+bc-4c^2}{2c^2} \\ 0 & 0 & 0 & -\frac{a^3-3a^2b+3ab^2-b^3}{2ac^2-2bc^2-2c^3} \\ -c & 0 & a-b+2c - \frac{c(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} + \frac{a^2-2ab-ac+b^2+bc-4c^2}{2c} & 1 \\ 0 & 0 & -\frac{c(a^3-3a^2b+3ab^2-b^3)}{2ac^2-2bc^2-2c^3} & 0 \end{bmatrix}$$

on voit que Sympy n'as pas pris en compte les cas potentiel où $\det(Q) = 0$. Pour lui les symboles a, b, c sont indépendants. Il faudra mener une discussion sur les valeurs de ces paramètres pour lesquelles $\det(Q) = 0$.

[83]: `Q.det().factor()`

[83]:
$$\frac{(a-b)^6}{4c^2(a-b-c)^2}$$

Trois cas notables se dégagent donc : $a = b$, $c = 0$ ou $a - b - c = 0$. Les deux derniers sont ajoutés parce $\det(Q)$ n'est pas défini dans ces cas et il faudrait examiner ce qui se produit dans ces situations.

Cas où $b = a$

[95]: `A1=A.subs({b:a}) ; display(A1)`

$$\begin{bmatrix} a-2c & 3c & 4c & -3c \\ 0 & a & 0 & 0 \\ -c & c & a+2c & -c \\ 0 & c & 0 & a \end{bmatrix}$$

[97]: `P1,T1=A1.jordan_form() ; display(P1,T1)`

$$\begin{bmatrix} 2c^3 & -5c^2 & 3c & 0 \\ 0 & 0 & 0 & 1 \\ c^3 & -2c^2 & c & 0 \\ 0 & 0 & c & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 1 & 0 \\ 0 & 0 & a & 1 \\ 0 & 0 & 0 & a \end{bmatrix}$$

Si $a = b$, la réduite de A est donc

$$T1 = \begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 1 & 0 \\ 0 & 0 & a & 1 \\ 0 & 0 & 0 & a \end{bmatrix}$$

Elle est distincte de la matrice T lorsqu'on applique la condition $b = a$. En fait les deux matrices $T1$ et $T1' = T|_{b=c}$ ne sont pas semblables puisque $\text{rg}(T1 - aI) = 3$ et $\text{rg}(T1' - aI) = 2$.

Cas où $c = 0$

[100]: `A2=A.subs({c:0}) ; display(A2)`

$$\begin{bmatrix} a & -a+b & 0 & a-b \\ 0 & b & 0 & 0 \\ 0 & 0 & a & 0 \\ 0 & 0 & 0 & b \end{bmatrix}$$

[101]: `Q2,T2=A2.jordan_form() ; display(Q2,T2)`

$$\begin{bmatrix} 1 & 0 & -\frac{-a+b}{a-b} & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 0 \\ 0 & 0 & 0 & b \end{bmatrix}$$

Encore un cas qui n'est pas un cas particulier de la forme générale (Q, T) . Lorsque $c = 0$, A est diagonalisable. Ce n'est pas le cas lorsque on reporte la condition $c = 0$ dans les matrice T originale.

Cas où $a - b - c = 0$

[102]: `A3=A.subs({c:a-b}) ; display(A3)`

$$\begin{bmatrix} -a+2b & 2a-2b & 4a-4b & -2a+2b \\ 0 & b & 0 & 0 \\ -a+b & a-b & 3a-2b & -a+b \\ 0 & a-b & 0 & b \end{bmatrix}$$

[105]: `Q3,T3=A3.jordan_form() ; display(Q3,T3)`

$$\begin{bmatrix} -2a+2b & 1 & 2a-2b & 0 \\ 0 & 0 & 0 & 1 \\ -a+b & 0 & a-b & 0 \\ 0 & 0 & a-b & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 1 \\ 0 & 0 & 0 & b \end{bmatrix}$$

[107]: `Q3.det().factor()`

[107]: $(a-b)^2$

[108]: `A4=A3.subs({b:a}) ; display(A4)`

$$\begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & a & 0 \\ 0 & 0 & 0 & a \end{bmatrix}$$

Finalement la condition $a - b - c = 0$ ne produit pas de cas à part entière.

Pour résumer : + si $a \neq b$ et $c \neq 0$ alors A est semblable à $\begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 1 \\ 0 & 0 & 0 & b \end{bmatrix}$; + si $c = 0$ alors A est semblable à $\begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & b & 0 \\ 0 & 0 & 0 & b \end{bmatrix}$; + si $a = b$ et $c \neq 0$ alors A est semblable à $\begin{bmatrix} a & 1 & 0 & 0 \\ 0 & a & 1 & 0 \\ 0 & 0 & a & 1 \\ 0 & 0 & 0 & a \end{bmatrix}$.

3.0.4 Exemple 5 : calcul d'une base du commutant d'une matrice

```
[605]: A=randMatrix(5,5,-1,2) ; A
```

```
[605]:
```

$$\begin{bmatrix} 2 & 2 & 0 & -1 & -1 \\ -1 & 0 & 2 & 1 & 2 \\ 1 & -1 & 1 & 2 & -1 \\ 1 & 0 & 2 & 1 & 2 \\ 0 & -1 & 0 & 2 & -1 \end{bmatrix}$$

Le bloc de code suivant permet de trouver la forme générale des matrices qui commutent avec A .

```
[606]: X=MatrixSymbol('x',5,5).as_explicit()
solX=solve(A*X-X*A,X)
Acom=X.subs(solX)
Acom
```

```
[606]:
```

$$\begin{bmatrix} -\frac{178x_{4,0}}{107} - \frac{27x_{4,1}}{107} + \frac{247x_{4,2}}{107} + \frac{147x_{4,3}}{107} + x_{4,4} & -\frac{226x_{4,0}}{107} - \frac{544x_{4,1}}{107} + \frac{118x_{4,2}}{107} - \frac{165x_{4,3}}{107} & \frac{68x_{4,0}}{107} + \frac{56x_{4,1}}{107} - \frac{9x_{4,2}}{107} + \frac{194x_{4,3}}{107} + \frac{214x_{4,4}}{107} \\ -\frac{27x_{4,0}}{107} + \frac{689x_{4,1}}{107} - \frac{105x_{4,2}}{107} + \frac{291x_{4,3}}{107} & -\frac{544x_{4,0}}{107} + \frac{729x_{4,1}}{107} + \frac{179x_{4,2}}{107} + \frac{418x_{4,3}}{107} + x_{4,4} & \frac{56x_{4,0}}{107} - \frac{42x_{4,1}}{107} + \frac{19x_{4,2}}{107} + \frac{37x_{4,3}}{107} + \frac{214x_{4,4}}{107} \\ \frac{247x_{4,0}}{107} - \frac{105x_{4,1}}{107} + \frac{7x_{4,2}}{107} + \frac{x_{4,3}}{107} & \frac{118x_{4,0}}{107} + \frac{179x_{4,1}}{107} - \frac{283x_{4,2}}{107} + \frac{36x_{4,3}}{107} & -\frac{9x_{4,0}}{107} + \frac{194x_{4,1}}{107} + \frac{37x_{4,2}}{107} + \frac{214x_{4,3}}{107} + \frac{214x_{4,4}}{107} \\ \frac{147x_{4,0}}{107} + \frac{291x_{4,1}}{107} + \frac{x_{4,2}}{107} + \frac{199x_{4,3}}{107} & -\frac{165x_{4,0}}{107} + \frac{418x_{4,1}}{107} + \frac{36x_{4,2}}{107} + \frac{209x_{4,3}}{107} & \frac{28x_{4,0}}{107} + \frac{86x_{4,1}}{107} + \frac{214x_{4,2}}{107} + \frac{214x_{4,3}}{107} + \frac{214x_{4,4}}{107} \\ x_{4,0} & x_{4,1} & x_{4,2} \end{bmatrix}$$

Le problème maintenant est comment former une base de l'ensemble de ses matrices. Cela passe par une identification des coefficients $x[i,j]$ qui sont encore présents dans la matrice 'Acom'. La tâche est rendu encore plus difficile si on veut écrire une fonction qui automatise toute la procédure.

Une idée simple pourrait être de dériver la matrice solution 'Acom' par rapport à chaque coefficient de 'X'. Une base de l'ensemble des solutions serait alors formée de toutes les matrices obtenues qui sont non nulles

```
[607]: Acom.diff(X[4,1])
```

```
[607]:
```

$$\begin{bmatrix} -\frac{27}{107} & -\frac{544}{107} & \frac{56}{107} & \frac{487}{107} & \frac{45}{107} \\ \frac{689}{107} & \frac{729}{107} & -\frac{42}{107} & -\frac{499}{107} & -\frac{114}{107} \\ \frac{107}{105} & \frac{179}{107} & \frac{194}{107} & -\frac{44}{107} & -\frac{39}{107} \\ -\frac{107}{291} & \frac{107}{418} & \frac{107}{86} & -\frac{107}{196} & -\frac{107}{50} \\ \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & -\frac{107}{107} & \frac{107}{107} \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

```
[610]: BaseCom=[]
for C in X :
    B_tmp=Acom.diff(C)
    if not B_tmp.is_zero_matrix :
        display(B_tmp)
        BaseCom.append(B_tmp)
print('la base contient', len(BaseCom), 'matrices')
```

$$\begin{bmatrix} -\frac{178}{107} & \frac{226}{107} & \frac{68}{107} & -\frac{150}{107} & \frac{47}{107} \\ -\frac{27}{107} & -\frac{544}{107} & \frac{56}{107} & \frac{487}{107} & \frac{45}{107} \\ \frac{247}{107} & \frac{118}{107} & -\frac{9}{107} & -\frac{84}{107} & -\frac{55}{107} \\ \frac{107}{147} & \frac{107}{165} & -\frac{107}{28} & \frac{107}{190} & -\frac{107}{31} \\ \frac{107}{107} & -\frac{107}{107} & \frac{107}{107} & \frac{107}{107} & -\frac{107}{107} \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -\frac{27}{107} & -\frac{544}{107} & \frac{56}{107} & \frac{487}{107} & \frac{45}{107} \\ \frac{689}{107} & \frac{729}{107} & -\frac{42}{107} & -\frac{499}{107} & -\frac{114}{107} \\ \frac{107}{105} & \frac{107}{179} & -\frac{107}{194} & -\frac{107}{44} & -\frac{107}{39} \\ -\frac{107}{291} & \frac{107}{418} & \frac{107}{86} & -\frac{107}{196} & -\frac{107}{50} \\ \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & -\frac{107}{107} & \frac{107}{107} \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \frac{247}{107} & \frac{118}{107} & -\frac{9}{107} & -\frac{84}{107} & -\frac{55}{107} \\ -\frac{107}{105} & \frac{107}{179} & -\frac{107}{194} & -\frac{107}{44} & -\frac{107}{39} \\ -\frac{107}{7} & \frac{107}{283} & \frac{107}{37} & \frac{107}{381} & -\frac{107}{119} \\ \frac{214}{1} & -\frac{214}{36} & \frac{214}{204} & \frac{214}{85} & \frac{107}{73} \\ \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & -\frac{107}{107} \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \frac{147}{107} & -\frac{165}{107} & \frac{28}{107} & \frac{190}{107} & -\frac{31}{107} \\ \frac{291}{107} & \frac{418}{107} & \frac{86}{107} & -\frac{196}{107} & \frac{50}{107} \\ \frac{107}{1} & \frac{107}{36} & \frac{107}{204} & -\frac{107}{85} & -\frac{107}{73} \\ \frac{107}{199} & \frac{107}{209} & \frac{107}{150} & \frac{107}{9} & -\frac{107}{132} \\ \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & \frac{107}{107} & \frac{107}{107} \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

la base contient 5 matrices

Exercice : Écrire une fonction Python qui implémente ces idées. _____

3.0.5 Exemple 6 : évaluation d'un polynôme en une valeur matricielle

Bizarrement, SymPy ne propose aucune fonction ou méthode pour évaluer un polynôme P en une matrice carrée M (ie calculer $P(M)$). On se propose de combler cette lacune en définissant une méthode qui s'utilise sous la forme `P.eval_matrix(M)` lorsque 'P' a été déclaré comme un `Polyet` 'M' comme une matrice carrée.

```
[ ]: def poly_eval_matrix(self, M): # self sera remplacé par l'instance de l'objet
    ↪ qui fait appel à la méthode.
    if not M.is_square: # tester si la matrice est carrée
        raise NonSquareMatrixError

    if (len(self.gens) != 1): # teste si le polynôme a été déclaré avec une
    ↪ seule indéterminée
        raise ValueError("un polynôme a une seule indéterminée est attendu")
```



```

    expr = self.as_expr(*P.gens) # convertit le polynôme en une expression
    const, non_const = expr.as_independent(self.gens[0]) # récupère la partie
    ↪ constante et la partie variable du polynôme

    return non_const.subs(self.gens[0], M).doit() + const * eye(M.rows)

Poly.eval_matrix = poly_eval_matrix # c'est tout bête d'ajouter une méthode à
    ↪ un objet existant

```

On teste maintenant sur plusieurs exemples.

Sur une matrice diagonale :

```

[496]: P=poly((x+1)**2,x)
M=diag(*symbols('a:3'))
display(P,M)
P.eval_matrix(M)

```

$\text{Poly}(x^2 + 2x + 1, x, \text{domain} = \mathbb{Z})$

$$\begin{bmatrix} a_0 & 0 & 0 \\ 0 & a_1 & 0 \\ 0 & 0 & a_2 \end{bmatrix}$$

```

[496]: 
$$\begin{bmatrix} a_0^2 + 2a_0 + 1 & 0 & 0 \\ 0 & a_1^2 + 2a_1 + 1 & 0 \\ 0 & 0 & a_2^2 + 2a_2 + 1 \end{bmatrix}$$


```

Sur une matrice compagne : on vérifie le théorème de Cayley-Hamilton pour une matrice compagne quelconque de taille 5.

```

[493]: a=symbols('a:5')
P=poly(x**5+sum(a[k]*x**k for k in range(5)),x)
C=Matrix.hstack(banded({-1:(1,1,1,1)}), -Matrix(5,1,a))
display(P,C)

```

$\text{Poly}(x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0, x, \text{domain} = \mathbb{Z}[a_0, a_1, a_2, a_3, a_4])$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & -a_0 \\ 1 & 0 & 0 & 0 & -a_1 \\ 0 & 1 & 0 & 0 & -a_2 \\ 0 & 0 & 1 & 0 & -a_3 \\ 0 & 0 & 0 & 1 & -a_4 \end{bmatrix}$$

```

[494]: P.eval_matrix(C)

```

```

[494]: 
$$\begin{bmatrix} 0 & 0 & -a_0a_3 + a_0a_4^2 - a_0(-a_3 + a_4^2) & -a_0a_3a_4 + a_0a_4^3 + a_4(a_0a_3 - a_0a_4^2) & 0 \\ 0 & 0 & a_0a_4 - a_1a_3 - a_1(-a_3 + a_4^2) + a_4(-a_0 + a_1a_4) & a_0a_3 - a_0a_4^2 - 2a_1a_3a_4 + a_1a_4^3 + a_3(-a_0 + a_1a_4) + a_4(a_0a_3 - a_0a_4^2) & 0 \\ 0 & 0 & a_1a_4 - a_2a_3 - a_2(-a_3 + a_4^2) + a_4(-a_1 + a_2a_4) & a_0a_4 + a_1a_3 - a_1a_4^2 - 2a_2a_3a_4 + a_2a_4^3 + a_3(-a_1 + a_2a_4) + a_4(a_0a_4 + a_1a_3 - a_1a_4^2) & 0 \\ 0 & 0 & a_2a_4 - a_3^2 - a_3(-a_3 + a_4^2) + a_4(-a_2 + a_3a_4) & a_1a_4 + a_2a_3 - a_2a_4^2 - 2a_3^2a_4 + a_3a_4^3 + a_3(-a_2 + a_3a_4) + a_4(a_1a_4 + a_2a_3 - a_2a_4^2) & 0 \\ 0 & 0 & 0 & a_2a_4 + a_3^2 - 3a_3a_4^2 + a_3(-a_3 + a_4^2) + a_4^4 + a_4 & 0 \end{bmatrix}$$


```

```
[495]: simplify(P.eval_matrix(C)) # Mission accomplie
```

```
[495]: 
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

```

3.0.6 Exemple 7 : décomposition de Dünford

On veut déterminer la décomposition de Dünford de la matrice

$$A = \begin{bmatrix} 6 & -5 & -2 & -1 & 3 \\ 5 & -4 & -2 & -1 & 3 \\ -3 & 3 & 1 & 1 & -1 \\ -1 & 0 & 1 & 0 & -1 \\ -2 & 1 & 1 & -1 & 0 \end{bmatrix}$$

```
[576]: A=Matrix([
[ 6, -5, -2, -1, 3],
[ 5, -4, -2, -1, 3],
[-3, 3, 1, 1, -1],
[-1, 0, 1, 0, -1],
[-2, 1, 1, -1, 0]])
```

```
[577]: A.charpoly().as_expr().factor()
```

```
[577]:  $(\lambda - 1)^3 (\lambda^2 + 1)$ 
```

On peut utiliser `jordan_form` pour trigonaliser A et ensuite construire les composantes D et N facilement

```
[578]: P,T=A.jordan_form()
display(P,T)
```

$$\begin{bmatrix} -\frac{1}{2} & \frac{1}{2} & 2 & -1+i & -1-i \\ -\frac{1}{2} & \frac{1}{2} & \frac{3}{2} & -1+i & -1-i \\ \frac{1}{2} & -\frac{3}{2} & 1 & 0 & 0 \\ \frac{1}{2} & -1 & 0 & 1 & 1 \\ \frac{1}{2} & -\frac{3}{2} & 0 & 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -i & 0 \\ 0 & 0 & 0 & 0 & i \end{bmatrix}$$

et récupérer ensuite D et N avec :

```
[579]: D=P*diag(1,1,1,-I,I)*P.inv()
N=A-D
```

```
display(D,N)
```

$$\begin{bmatrix} 6+i(-1-i)(1-\frac{3i}{2})-i(-1+i)(1+\frac{3i}{2}) & -6-i(-1-2i)(-1+i)+i(-1-i)(-1+2i) & -1+\frac{i(-1+i)}{2}-\frac{i(-1-i)}{2} \\ 5+i(-1-i)(1-\frac{3i}{2})-i(-1+i)(1+\frac{3i}{2}) & -5-i(-1-2i)(-1+i)+i(-1-i)(-1+2i) & -1+\frac{i(-1+i)}{2}-\frac{i(-1-i)}{2} \\ 0 & 0 & 1 \\ -2-i(1+\frac{3i}{2})+i(1-\frac{3i}{2}) & 2+i(-1+2i)-i(-1-2i) & 1 \\ -2-i(1+\frac{3i}{2})+i(1-\frac{3i}{2}) & 2+i(-1+2i)-i(-1-2i) & 1 \\ i(-1+i)(1+\frac{3i}{2})-i(-1-i)(1-\frac{3i}{2}) & 1-i(-1-i)(-1+2i)+i(-1-2i)(-1+i) & -1+\frac{i(-1-i)}{2}-\frac{i(-1+i)}{2} \\ i(-1+i)(1+\frac{3i}{2})-i(-1-i)(1-\frac{3i}{2}) & 1-i(-1-i)(-1+2i)+i(-1-2i)(-1+i) & -1+\frac{i(-1-i)}{2}-\frac{i(-1+i)}{2} \\ -3 & 3 & 0 \\ 1-i(1-\frac{3i}{2})+i(1+\frac{3i}{2}) & -2+i(-1-2i)-i(-1+2i) & 0 \\ -i(1-\frac{3i}{2})+i(1+\frac{3i}{2}) & -1+i(-1-2i)-i(-1+2i) & 0 \end{bmatrix}$$

Puisque la décomposition de Dünford est unique et que la matrice de départ A est réelle alors N et D sont en fait réelles. En effet

$$A = \bar{A} \implies \bar{D} - D = N - \bar{N}$$

On sait que $\bar{D}$ et $\bar{N}$ sont des polynômes en $\bar{A}$ et donc en A . Elle commutent donc respectivement avec D et N qui sont elles aussi des polynômes en A . On en déduit que $\bar{D} - D$ est diagonalisable et $N - \bar{N}$ est nilpotente. Ces deux matrices sont égales donc elles sont nulles. Soit $\bar{D} = D$ et $\bar{N} = N$.

Dans notre exemple, il suffit juste de forcer une simplification.

```
[580]: D=simplify(P*diag(1,1,1,-I,I)*P.inv())
N=A-D
display(D,N)
```

$$\begin{bmatrix} 5 & -4 & -2 & 0 & 2 \\ 4 & -3 & -2 & 0 & 2 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & -2 & 1 & -1 & 0 \\ 1 & -2 & 1 & -2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -1 & 0 & -1 & 1 \\ 1 & -1 & 0 & -1 & 1 \\ -3 & 3 & 0 & 1 & -1 \\ -2 & 2 & 0 & 1 & -1 \\ -3 & 3 & 0 & 1 & -1 \end{bmatrix}$$

```
[581]: D+N == A
```

```
[581]: True
```

```
[582]: D.is_diagonalizable() # is_diagonalisable is a method, hence the parenthesis
```

```
[582]: True
```

```
[583]: (N**5).is_zero_matrix # is_zero_matrix is a property
```

[583] : True

Exercice : écrire une méthode `dunford` attachée à la classe `Matrix` qui retourne les matrices D et N en utilisant `jordan_form`. _____

Qu'en est-il maintenant de la méthode vue en devoir libre et qui permet de calculer D et N comme des polynômes en A .

Sachant que $\chi_A = (X - i)(X + i)(X - 1)^3$. Considérons des polynômes W_1 et W_2 et W_3 tels que

$$W_1 Q_1 + W_2 Q_2 + W_3 Q_3 = 1$$

$$\text{avec } Q_1 = (X + i)(X - 1)^3 \quad Q_2 = (X - i)(X - 1)^3 \quad Q_3 = (X - i)(X + i) = X^2 + 1$$

On a démontré que

$$D = i \cdot W_1(A)Q_1(A) - i \cdot W_2(A)Q_2(A) + 1 \cdot W_3(A)Q_3(A)$$

Définissons les polynômes Q_1 , Q_2 et Q_3

```
[584] : P1=x-I
P2=x+I
P3=(x-1)**3
Q1=poly(P2*P3,x)
Q2=poly(P1*P3,x)
Q3=poly(P1*P2,x)
display(Q1,Q2,Q3)
```

Poly($x^4 + (-3 + i)x^3 + (3 - 3i)x^2 + (-1 + 3i)x - i, x, domain = \mathbb{ZZ}_I$)

Poly($x^4 + (-3 - i)x^3 + (3 + 3i)x^2 + (-1 - 3i)x + i, x, domain = \mathbb{ZZ}_I$)

Poly($x^2 + 1, x, domain = \mathbb{ZZ}_I$)

La fonction `gcdex` qui calcule le PGCD et des coefficients de Bezout ne prend en charge que deux arguments. Mais il est possible de l'utiliser de façon récursive pour calculer ces coefficients pour plus que 2 polynômes.

```
[596] : U1,U2,gcd12=gcdex(Q1,Q2)
display(U1,U2,gcd12)
```

Poly($-\frac{i}{2}, x, domain = \mathbb{QQ}_I$)

Poly($\frac{i}{2}, x, domain = \mathbb{QQ}_I$)

Poly($x^3 - 3x^2 + 3x - 1, x, domain = \mathbb{QQ}_I$)

```
[597] : V1,V2,_=gcdex(gcd12,Q3) # on jette à la poubelle le troisième élément du tuple
↪avec _
display(V1,V2)
```

$$\text{Poly}\left(-\frac{1}{4}x + \frac{1}{4}, x, \text{domain} = \text{QQ_I}\right)$$

$$\text{Poly}\left(\frac{1}{4}x^2 - x + \frac{5}{4}, x, \text{domain} = \text{QQ_I}\right)$$

[603]: `V1*U1*Q1+V1*U2*Q2+V2*Q3 # vérification`

[603]: `Poly(1, x, domain = QQ_I)`

[604]: `W1=V1*U1
W2=V1*U2
W3=V2
display(W1,W2,W3)`

$$\text{Poly}\left(\frac{i}{8}x - \frac{i}{8}, x, \text{domain} = \text{QQ_I}\right)$$

$$\text{Poly}\left(-\frac{i}{8}x + \frac{i}{8}, x, \text{domain} = \text{QQ_I}\right)$$

$$\text{Poly}\left(\frac{1}{4}x^2 - x + \frac{5}{4}, x, \text{domain} = \text{QQ_I}\right)$$

[600]: `P=I*W1*Q1-I*W2*Q2+W3*Q3
display(P)`

$$\text{Poly}\left(-\frac{1}{4}x^5 + \frac{5}{4}x^4 - \frac{5}{2}x^3 + \frac{5}{2}x^2 - \frac{5}{4}x + \frac{5}{4}, x, \text{domain} = \text{QQ_I}\right)$$

On utilise maintenant la méthode `eval_matrix` définie dans l'exemple précédent pour former la matrice D .

[601]: `D1=P.eval_matrix(A)
N1=A-D1`

[602]: `D1 == D`

[602]: `True`

Exercice : écrire une fonction Python qui calcule des coefficients de Bezout pour une famille quelconque de polynômes non nuls. L'utiliser pour écrire une fonction qui automatise la procédure précédente et retourne le polynôme P et les matrices D et N . _____